Computerforensik - Dateisysteme

Dr. Stefan Hahndel

Technische Hochschule Ingolstadt
Fakultät für Elektrotechnik und Informatik

Gliederung der Teile 1-3

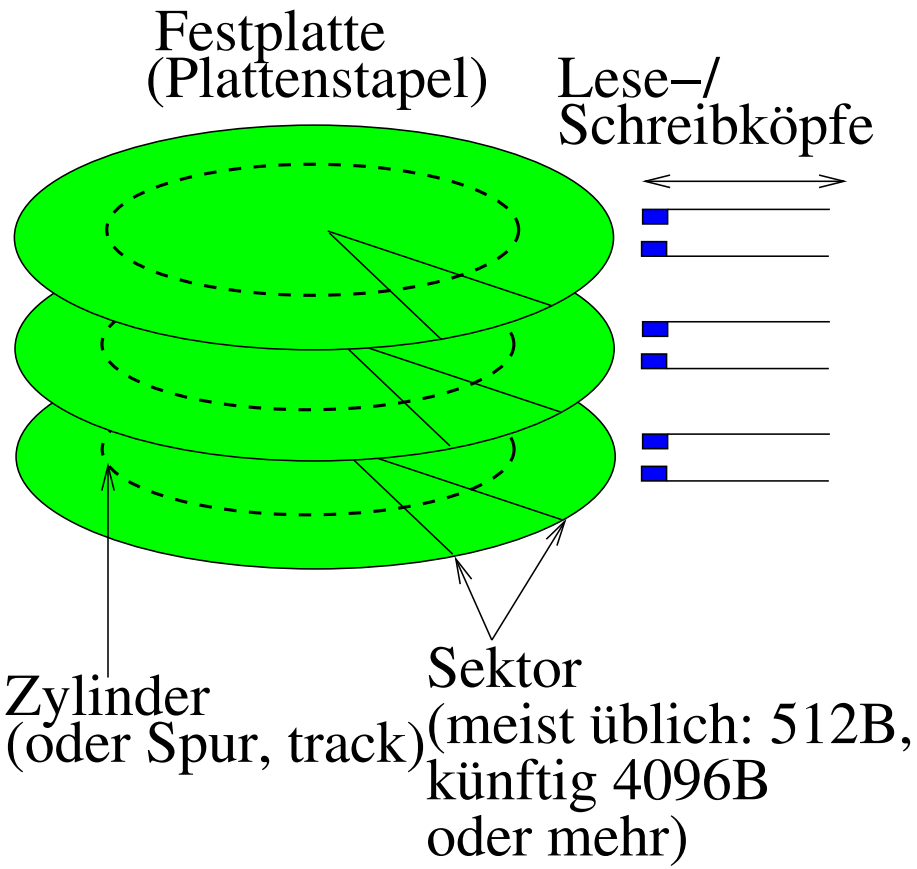
1. Datenträger und Plattenaufbau

2. Dateisysteme
   - Das FAT-Dateisystem
   - Dateisysteme unter Unix/Linux an den Beispielen Ext2-4 / ReiserFS
   - NTFS - New Technology File System

Teil I

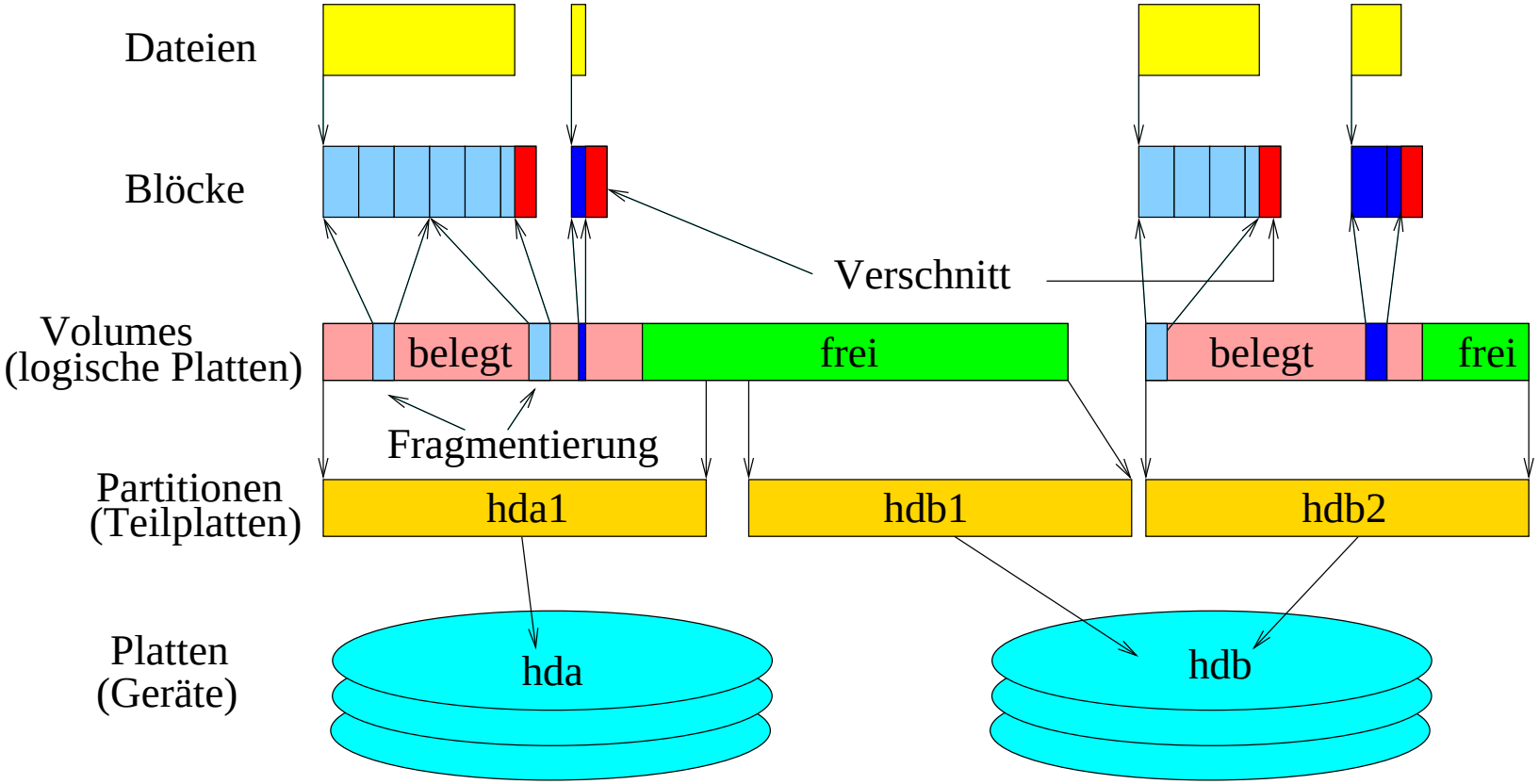
Datenträger,Dateisysteme und Tools

Plattenaufbau



- Heute meist LBA (Logical Block Adressing)
- Plattengemetrie versteckt
- Systemsicht: Reihung von Blöcken
- Integriertes Defektmanagement

Abstraktionsebenen Dateiorganisation



Realisierung Dateiorganisation

- Datei durch Verzeichniseintrag (Pfadname)
- Block
  - Allokationseinheit, logischer Sektor, Cluster
  - 0 .. n (relativ in Volume bzw. Partition)
- Volume: Eintrag im Volume-Manager (BS)
- Partition: Eintrag im Partitionstabelle (im Bootblock)
- Sektor (= Plattensektor, Plattenblock): Zylinder/Sektor/Kopf oder heute meist 0 .. n (bei LBA)
- Bootblock am Beginn der Platte bzw. Partition
- Dateisystem: Strukturierung der logischen Blöcke (Behandlung später)

Slackspace

Was ist Slackspace?

Als Slackspace wird im allgemeinen der Bereich auf einem Datenträger bezeichnet, der nicht durch irgendwelche Verwaltungsstrukturen des Betriebssystems oder von den Filesystemen benutzt wird.

- Slackspace wird daher gern zum „verstecken“ von Daten missbraucht
- Daten werden daher mit vom „normalen Nutzer“ verwendeten Werkzeugen (z.B. Windows Browser, dir, ls etc.) nicht angezeigt
- Inzwischen gibt es sogar - z.B. unter Linux - eigene Tools die sich das für versteckte Dateisysteme zu Nutze machen

Beispiel: Hidden Protected Area (HPA) (1)

Was ist HPA?

Die HPA, manchmal auch als Host Protected Area bezeichnet, ist ein Bereich am Ende der Festplatte, der durch das Betriebssystem normalerweise nicht sichtbar ist.

- Zuerst beim IBM Thinkpad eingeführt, inzwischen Standard (seit ATA-5), auf fast allen modernen Platten verfügbar
- Dazu gibt es einige neue IDE-Kommandos
  - `IDENTIFY_DEVICE` (normalerweise genutzt)
  - `SET_MAX_ADDRESS`
  - `READ_NATIVE_MAX_ADDRESS`

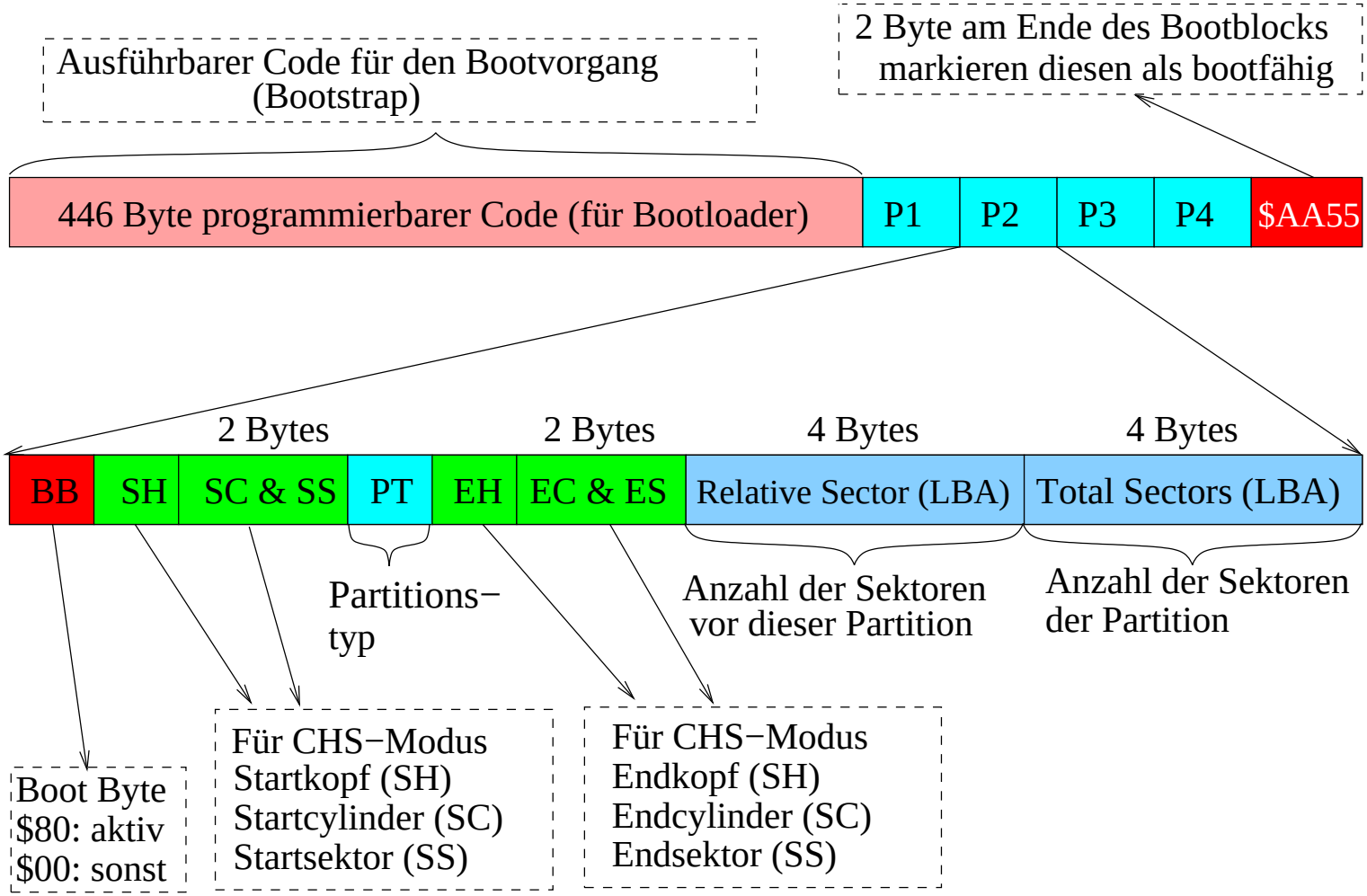
Beispiel: Hidden Protected Area (HPA) (2)

- Normalerweise ist die max. Adresse schon von Werk aus verändert
- Bereich enthält oft Rettungstools, ein Windows-Backup für den Restore u.ä.
- Zugriff von normalen BS aus nur mit speziellen „Low-Level-Tools“
- z.B. mit `dd` die entsprechenden Sektoren woanders hin kopieren und analysieren
- Nutzung des FUSE-Dateisystems hpafs, derzeit noch Version 0.1
- Nutzung der HPA-Tools

Wo ist Slackspace zu finden?

- Auf der untersten Ebene (Partitionen):
  - In Partitionslücken oder versteckten Partitionen
  - In Swappartitionen (nur bei sehr grossem Hauptspeicher verwendbar)
- Auf Dateisystemebene
  - In nicht allokierten Blöcken
  - In unvollständig gefüllten Blöcken
  - In ungenutzten Verwaltungsbereichen des Dateisystems
  - In künstlich als defekt markierten Datenblöcken
- Auf Dateiebene
  - z.B. An reguläre Dateien hinten angehängt
  - Dateisysteme in Dateien
  - Noch „exotischere Orte“: z.B. im ELF-Header ;-)

Aufbau Master Boot Record - LB0



Wichtige Partitionskennungen

00 Leer	01 FAT12	04 FAT16 <32M
05 Erweiterte	06 FAT16	07 HPFS/NTFS
08 AIX	09 AIX bootfähig	0b W95 FAT32
0c W95 FAT32 (LBA)	0e W95 FAT16 (LBA)	0f W95 Erw. (LBA)
11 Verst. FAT12	14 Verst. FAT16 <32	16 Verst. FAT16
17 Verst. HPFS/NTFS	1b Verst. W95 FAT32	1c Verst. W95
1e Verst. W95 FAT16	39 Plan 9	3c PartitionMagic
63 SysV	64 Novell Netware	65 Novell Netware
82 Linux Swap	83 Linux	85 Linux erweitert
86 NTFS volume set	87 NTFS volume set	8e Linux LVM
a5 FreeBSD	a6 OpenBSD	a7 NeXTSTEP
a8 Darwin UFS	a9 NetBSD	ab Darwin boot
be Solaris boot	bf Solaris BeOS fs	fd Linux raid

Zerstörte Partitionstabellen mit gpart raten

- Einsetzbar, wenn primäre Partitionstabelle fehlt
- Partitionen vorhanden, aber BS hat keinen Zugriff drauf
- gpart ist ein Open Source Tool für Linux/Unix
- Versucht Partitionen auf der Harddisk zu erraten
  - ignoriert dabei ggf. vorhandene Partitionstabelle
  - scannt sektorweise Disk oder Imagefiles auf Partitionen/Dateisysteme
  - gpart bedient sich diverser Module zur Dateisystemerkennung
  - Module können selbst erstellt werden; das Modul muss dazu drei (im Readme beschriebene) C-Funktionen bereitstellen
- Auch auf Images (dd) einsetzbar!

Vorhandene Module in gpart

Modname	Typ	Description
fat	0x01	Primary DOS with 12 bit FAT
	0x04	Primary DOS with 16 bit FAT (<= 32MB)
	0x06	Primary 'big' DOS (> 32MB)
	0x0B	DOS or Windows 95 with 32 bit FAT
	0x0C	DOS or Windows 95 with 32 bit FAT, LBA
ntfs, hpfs	0x07	OS/2 HPFS, NTFS, QNX or Advanced UNIX
ext2	0x83	Linux ext2 filesystem
lswap	0x82	Linux swap
bsddl	0xA5	FreeBSD/NetBSD/386BSD
s86dl	0x82	Solaris/x86 disklabel
minix	0x80,0x81	Minix V1 bzw. V2
rfs	0x83	Reiser filesystem
hmlvm	0xFE	Linux LVM physical volumes
qnx4	0x4F	QNX 4.x
beos	0xEB	BeOS fs
xfs	0x83	SGI XFS filesystem

Beispielhafte Ausgabe von gpart

```
Possible partition(DOS FAT), size(6000mb), offset(0mb)
   type: 012(0x0C)(DOS or Windows 95 with 32 bit FAT, LBA)
   size: 6000mb #s(12289662) s(63-12289724)
   chs:  (0/1/1)-(764/254/63)d (0/1/1)-(764/254/63)r
   hex:  00 01 01 00 0C FE BF FC 3F 00 00 00 7E 86 BB 00

Possible extended partition at offset(18402mb)
   Possible partition(DOS FAT), size(13005mb), offset(18402mb)
      type: 012(0x0C)(DOS or Windows 95 with 32 bit FAT, LBA)
      size: 13005mb #s(26635706) s(37688553-64324258)
      chs:  (1023/254/63)-(1023/254/63)d (2346/1/1)-(4003/254/62)r
      hex:  00 FE FF FF 0C FE FF FF E9 14 3F 02 BA 6D 96 01

   Possible partition(Linux swap), size(502mb), offset(42931mb)
      type: 130(0x82)(Linux swap or Solaris/x86)
      size: 502mb #s(1028096) s(87923808-88951903)
      chs:  (1023/254/63)-(1023/254/63)d (5473/1/1)-(5536/254/62)r
      hex:  00 FE FF FF 82 FE FF FF 60 9C 3D 05 00 B0 0F 00
...
```

Neuere Alternative: testdisk

- Verfügbar für DOS, Windows, Linux und MAC, Solaris oder als Boot-CD
- Funktionalität unter Solaris und MAC eingeschränkt
- Unterstützt die Dateisysteme FAT12/16/32, NTFS, ext2, ext3, BeFS, CramFS, HFS, HFS+, disklabel (BSD, Solaris), JFS, Linux Raid, Linux Swap, LVM, LVM2, NSS, ReiserFS 3.5, 3.6 und 4, UFS, UFS2 und XFS.
- Vorteile gegenüber gpart
  - Für Einsteiger einfacher Bedienbar (Cursorsteuerung)
  - Erkennt mehr in der Standardversion
- Bei Nutzung Logfile erstellen und signieren!

Startmenue

hahndel@chaoshs:/...Forensik/beispiele - Befehlsfenster 3 - Konsole

Sitzung Bearbeiten Ansicht Lesezeichen Einstellungen Hilfe

```
TestDisk 6.5, Data Recovery Utility, October 2006
Christophe GRENIER <grenier@cgsecurity.org>
http://www.cgsecurity.org

  TestDisk is free software, and
comes with ABSOLUTELY NO WARRANTY.

Select a media (use Arrow keys, then press Enter):
Disk /dev/hdb - 4294 MB / 4096 MiB (RO)
Disk /dev/hdc - 300 GB / 279 GiB
Disk /dev/hdd - 123 GB / 115 GiB
Disk /dev/sda - 250 GB / 232 GiB
Disk /dev/sdg - 15 GB / 14 GiB




[Proceed ]  [  Quit  ]

Note: Disk capacity must be correctly detected for a successful recovery.
If a disk listed above has incorrect size, check HD jumper settings, BIOSdetec
```

Befehlsfenster | Befehlsfenster 2 | Befehlsfenster 3

Beispielergebnis mit Testdisk

hahndel@chaoshs:~/tmp - Befehlsfenster 3 - Konsole

Sitzung Bearbeiten Ansicht Lesezeichen Einstellungen Hilfe

```
TestDisk 6.5, Data Recovery Utility, October 2006
Christophe GRENIER <grenier@cgsecurity.org>
http://www.cgsecurity.org

Disk /dev/sdg - 15 GB / 14 GiB - CHS 1869 255 63
     Partition               Start        End    Size in sectors
D Linux                    0   1  1  1868 254 63   30025422
D Linux                  123   0  1   245 254 63    1975995 [Forensik2]
D Linux                  246   0  1   368 254 63    1975995 [Forensik3]
D FAT32                  369   1  1   491 254 63    1975932 [Forensik5]
D Linux                  492   1  1  1221 254 63   11727387 [Forensik6]
D Linux                 1222   1  1  1868 254 63   10393992 [Forensik7]

Structure: Ok.  Use Up/Down Arrow keys to select partition.
Use Left/Right Arrow keys to CHANGE partition characteristics:
*=Primary bootable  P=Primary  L=Logical  E=Extended  D=Deleted
Keys A: add partition, L: load backup, T: change type, P: list files,
     Enter: to continue
EXT2 Sparse superblock, 5321 MB / 5075 MiB
```

Befehlsfenster | Befehlsfenster 2 | Befehlsfenster 3

GUID-Partition

Offset	Länge	Inhalt
0	8B	Signatur („EFI PART“, 45h 46h 49h 20h 50h 41h 52h 54h)
8	4B	Revision (00h 00h 01h 00h)
12	4B	Header-Größe - Little Endian (5Ch 00h 00h 00h entspricht 92 bytes)
16	4B	CRC32-Prüfsumme Header
20	4B	Reservierter Bereich - (0 . . . 0)
24	8B	Position des eigenen LBA (dieses Headers)
32	8B	Position des Backup-LBA (des anderen Headers)
40	8B	Erster benutzbarer LBA für Partitionseinträge (primäre Partitionstabelle letzter LBA + 1, normalerweise 34)
48	8B	Letzter benutzbarer LBA für Partitionseinträge (sekundäre Partitionstabelle letzter LBA - 1, normalerweise Datenträgergröße - 34)
56	16B	Datenträger-GUID (als Referenz siehe auch UUID bei UNIXe)
72	8B	Start-LBA der Partitionstabelle
80	4B	Anzahl der Partitionseinträge (Partitionen)
84	4B	Größe eines Partitionseintrags (normalerweise 128)
88	4B	Partitionstabellen-CRC32-Prüfsumme
92	*	Reservierter Bereich; muss mit Nullen, für den Rest des Blocks, belegt sein (420 Bytes bei einem 512-byte LBA)

Schema der GUID-Tabelle (LBA 64 Bit)

Klassischer MBR				LBA Nr. 0
Primärer GPT header				LBA Nr. 1
Eintrag 1	Eintrag 2	Eintrag 3	Eintrag 4	LBA Nr. 2
Einträge 5 - 128				LBA Nr. 3 - 33
Partition 1				LBA Nr. 34
Partition 2				
Restliche Partitionen				
⋮				
Eintrag 1	Eintrag 2	Eintrag 3	Eintrag 4	LBA -33
Einträge 5 - 128				LBA -2 bis -32
Sekundärer GPT header				LBA -1

GUID-Partition

Offset	Länge	Inhalt
0	16B	Partitionstyp-GUID
16	16B	Eindeutige Partitions-GUID
32	8B	Partitionsbeginn (erster LBA - Little-Endian)
40	8B	Partitionsende (letzter LBA - inclusive)
48	8B	Attribute
56	72B	Partitionsname (36 UTF-16LE-Zeichen)
$\sum$	128B	

Einige Beispiele für GUIDs

Partitionstyp	GUID
unbenutzte Partition	00000000-0000-0000-0000-000000000000
Master Boot Record	024DEE41-33E7-11D3-9D69-0008C781F39F
BIOS-Bootpartition	21686148-6449-6E6F-744E-656564454649
EFI Systempartition	C12A7328-F81F-11D2-BA4B-00A0C93EC93B
Intel Fast Flash (iFFS)	D3BFE2DE-3DAF-11DF-BA40-E3A556D89593
Windows Datenpartition	EBD0A0A2-B9E5-4433-87C0-68B6B72699C7
Linux Datenpartition	0FC63DAF-8483-4772-8E79-3D69D8477DE4
Windows Wiederherstellungsparti.	DE94BBA4-06D1-4D40-A16A-BFD50179D6AC
Windows LDM Daten	AF9B60A0-1431-4F62-BC68-3311714A69AD
Linux LVM	E6D6D379-F507-44C2-A23C-238F2A3DF928
Linux Swap	0657FD6D-A4AB-43C4-84E5-0933C84B4F4F
Apple HFS+	48465300-0000-11AA-AA11-00306543ECAC

Eigenschaften Dateisysteme

Dateisysteme unterscheiden sich durch

- maximale Größe der Dateien
- Größe der (logischen) Blöcke
- Strategie beim Zuteilen neuer Blöcke
- Struktur der Ortsbeschreibungen
- Umgang mit gelöschten Dateien
- Berechtigungssysteme / Zugriffsschutz

wichtige Dateisysteme

NTFS, UFS, FAT (mehrere Unterarten), HPFS, ext2-ext4, reiser(2-4)

Dateisystem FAT

- FAT = File Allocation Table
- FAT12: Auf Datenträgern und Partitionen bis 12MB (z.B. Disketten)
- FAT16: DOS, Windows 3.1; „8+3“-Namen, bis 2GB je Partition bei FAT16
- VFAT: Erweiterung der FAT ab Windows 95; wie FAT plus lange Dateinamen; wird vielseitig unterstützt (Windows NT, 2000, XP, Linux u.a.)
- FAT32: ab Windows 95b; effizienter bei grossen Platten (>2GB)
- Heute: Hauptsächlich Verwendung bei Speicherkarten und USB-Sticks

Aufbau FAT - Bootsektor/Bios Parameter Block (BPB)

Bootsektor/BPB	reservierte Sektoren	FAT 1	FAT 2	Wurzel–verzeichnis	Datenbereich

Offset	Länge	Inhalt
0x00	3	Sprunganweisung zum Boot-Code
0x03	8	OEM Name, z.B. MSDOS5.0.
0x0B	2	Bytes pro Sektor (512, 1024, 2048 oder 4096)
0x0D	1	Sektoren pro Cluster (Zweierpotenz zwischen 1 und 64)
0x0E	2	Anzahl reservierter Sektoren inkl. Bootsektor
0x10	1	Anzahl der FAT-Kopien (üblicherweise 2)
0x11	2	Maximum der Einträge im Wurzelverz. (bei FAT 32 ungenutzt)
0x13	2	Sektoranzahl des Mediums (max. 65535, bei großen Medien stattdessen 4-Byte-Wert bei 0x20. (bei FAT 32 ungenutzt)
0x15	1	Media Descriptor Byte (veraltet,ab W-Server 2003 unbeachtet, z.B. 0xF8 (Festplatte), 0xF9 (DS, 80T, 9S)
0x16	2	Sektorenanzahl je FAT. (Bei FAT 32 ungenutzt)
0x18	2	Sektoren pro Spur
0x1A	2	Anzahl der Seiten bzw. Schreib-Lese-Köpfe
0x1C	4	Anzahl der versteckten Sektoren vor dem Bootsektor.
0x20	4	Gesamtsektoranzahl, falls größer als 65535

Aufbau FAT - Fat12/16-BPB ab 0x24

Bootsektor/BPB	reservierte Sektoren	FAT 1	FAT 2	Wurzel−verzeichnis	Datenbereich

Offset	Länge	Inhalt
0x24	1	Phys. BIOSLaufwerksnummer (Disketten: 0x00, Festplatten: 0x80, 0x81, ...)
0x25	1	Reserviert (hier immer 0x00)
0x26	1	Erweiterte Bootsignatur
0x27	4	Dateisystems-ID (Seriennummer)
0x2B	11	Name des Dateisystems (max. 11 Zeichen, veraltet)
0x36	8	FAT-Variante, hier "FAT12 " oder "FAT16 "
0x3E	448	Maschinencode für Bootloader
0x1FE	2	BIOS-Bootsektorsignatur: 0xAA55 (bootfähig, Offset fest!)

Aufbau FAT - Fat32-BPB ab 0x24

Bootsektor/BPB	reservierte Sektoren	FAT 1	FAT 2	Wurzel−verzeichnis	Datenbereich

Offset	Länge	Inhalt
0x24	4	Anzahl der Sektoren pro FAT
0x28	2	FAT Flags
0x2A	2	FAT-32-Version (aktuell stets 0x0000)
0x2C	4	Clusternummer, an der das Wurzelverzeichnis beginnt
0x30	2	Sektornummer des FS Information Sector (normalerweise 1)
0x32	2	Sektornummer der Bootsektorkopie (Microsoft empfiehlt 0x0006)
0x34	12	Reserviert für Erweiterungen, aktuell immer 0.
0x40	1	Physische BIOS-Laufwerksnummer
0x41	1	Reserviert
0x42	1	Signatur
0x43	4	Dateisystems-ID (Seriennummer)
0x47	11	Name des Dateisystems (Ungenutzt)
0x52	8	FAT-Version, hier immer "FAT32 "
0x5A	420	Maschinencode des Bootloaders
0x1FE	2	BIOS-Bootsektorsignatur: 0xAA55 (bootfähig, Offset fest!)

Aufbau FAT - Die namensgebende FAT selbst

- Verkettung der Cluster als 32-Bit-Zahlen
- Startcluster kommt aus Verzeichniseintrag
- Listenende durch Wert -1 gekenntzeichnet
- Freie Einträge durch Wert 0 gekennzeichnet

Auszug aus FAT

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	...
0	**3**	**8**	**4**	**5**	**11**	**0**	**0**	**9**	**10**	**13**	**12**	**−1**	**−1**	**0**	**0**	

Clusterbelegung der 1. Datei

2	8	9	10	13

Clusterbelegung der 2. Datei

1	3	4	5	11	12

Aufbau FAT - Verzeichniseinträge

Offset	Länge	Inhalt
0x00	8	Dateiname vor dem . (erstes Byte ist Statusbyte)
0x08	3	Suffix des Dateinamen (nach dem .)
0x0B	1	Attribute (siehe nächste Tabelle)
0x0C	2	Reserviert
0x0E	4	Datum und Zeit der Erzeugung (als MSDOS time stamp)
0x12	2	Letztes Zugriffsdatum
0x14	2	2 high bytes des FAT32 Startclusters (=0 bei FAT12/16)
0x16	4	Datum und Zeit des letzten Schreibzugriffs
0x1A	2	2 low bytes des FAT32 Startclusters
0x1C	4	Dateigröße in Bytes, Wert 0 für Verzeichnis

Attribute:

Bitmuster	Beschreibung	Bitmuster	Beschreibung
0000 0001	nur lesender Zugriff	0000 1000	Volume Label
0000 0010	Versteckte Datei	0000 1111	Langer Dateiname
0000 0100	Systemdatei	0001 0000	Verzeichnis
0010 0000	Archiv		

Wiederherstellen gelöschter Dateien

- Falls Datei „nur“ gelöscht und noch keine Dateicluster überschrieben einfach, da Löschung nur durch „?“ im ersten Dateinamen gekennzeichnet
- Erweiterte Möglichkeiten bieten verschiedene Forensik-Werkzeuge wie Encase oder Sleuthkit/Autopsy
- Auf der Kommandozeile unter Linux/Unix: `fatback` erstellt von Nicholas Harbour vom DoD Computer Forensics Lab
- `fatback` bietet automatischen und manuellen Modus
- Bietet im manuellen Modus eine Art Kommandoshell:
  - Viele Shellkommandos entsprechen den Unix- und Doskommandos: `ls, dir, cp, copy, pwd`
  - Aber auch suchen und kopieren von Clusterketten möglich: `cpchain, chain`

Einführung in exFAT

- exFAT = Extended File Allocation Table
- 2006 von Microsoft eingeführt, optimiert für Flash-Speicher
- Kompromiss zwischen FAT32 (alt) und NTFS (komplex)
- Unterstützt große Dateien und Volumes

Warum exFAT?

- FAT32: Dateigröße auf 4 GiB begrenzt
- NTFS: komplex, hohe Overhead, patentiert
- exFAT:
  - leichtgewichtig
  - keine Beschränkung auf 4 GiB
  - kompatibel mit vielen Systemen (inkl. Linux, macOS, Android)

Technische Merkmale von exFAT

- Clustergrößen bis 32 MiB möglich
- Dateigröße: bis $2^{64} - 1$ Byte
- Max. Volumegröße: 128 PiB
- Keine Journaling-Funktion
- Benutzt eine Bitmap zur Cluster-Verwaltung

Vergleich: FAT32 vs. exFAT

	FAT32	**exFAT**
Max. Dateigröße	4 GiB	16 EiB (theoretisch)
Max. Volumegröße	2 TiB	128 PiB
Journaling	Nein	Nein
Unterstützt Zeitstempel	Teilweise	Ja (inkl. Zeitzone)
Kompatibilität	Hoch	Hoch

Verwendung von exFAT

- SDXC-Karten (ab 64 GiB Pflichtstandard)
- USB-Sticks, externe SSDs
- Industrielle Geräte mit Flash-Speicher
- Betriebssysteme: Windows, macOS, Linux (ab Kernel 5.4 nativ)

Interner Aufbau: Überblick

FAT32:

- Bootsektor
- FS-Info-Sektor
- FAT (mind. 2 Kopien)
- Root-Verzeichnis (fester Ort)
- Datenbereich

exFAT:

- Bootsektor + OEM Parameter
- Upcase-Tabelle
- Allocation Bitmap
- FAT (optional)
- Root-Dir = erste Datei

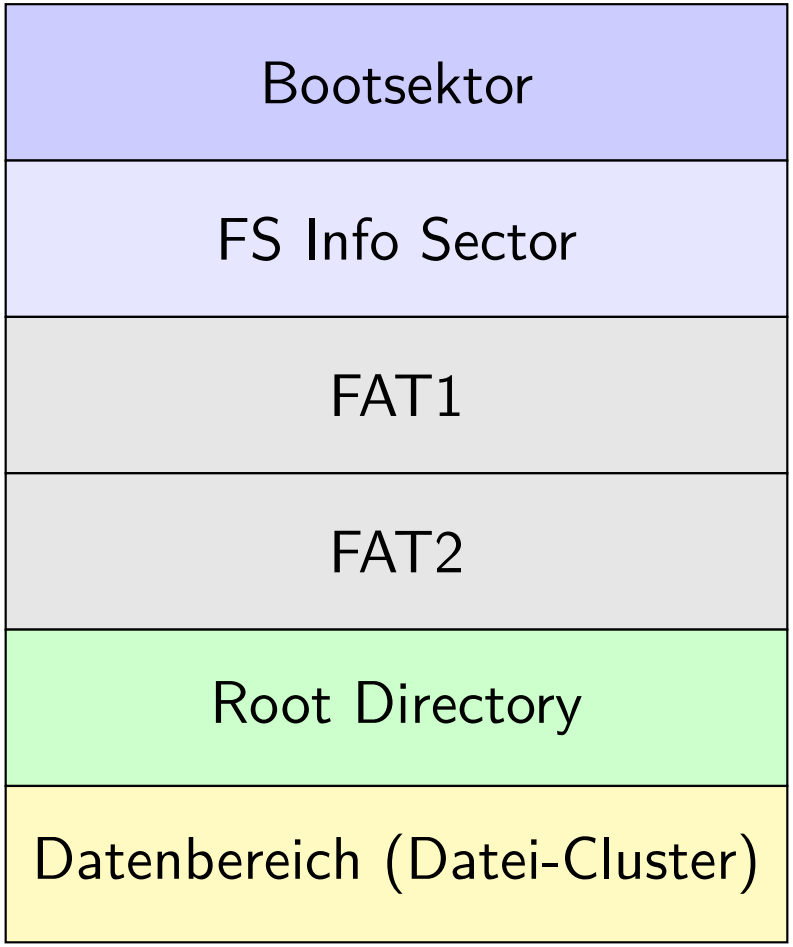
Wichtige Unterschiede im Aufbau

- **Root-Verzeichnis:**
  - FAT32: fest an einer Position, begrenzte Einträge
  - exFAT: als normale Datei realisiert, beliebig groß
- **Dateizuordnung:**
  - FAT32: ausschließlich über FAT-Tabelle
  - exFAT: Cluster-Kette optional (FAT oder Contiguous-Flag)
- **Verzeichniseinträge:**
  - FAT32: 8.3-Dateinamen, VFAT-Erweiterung
  - exFAT: strukturierte Eintragsgruppen, UTF-16
- **Zeitsysteme:**
  - exFAT: UTC mit Zeitzonenfeld

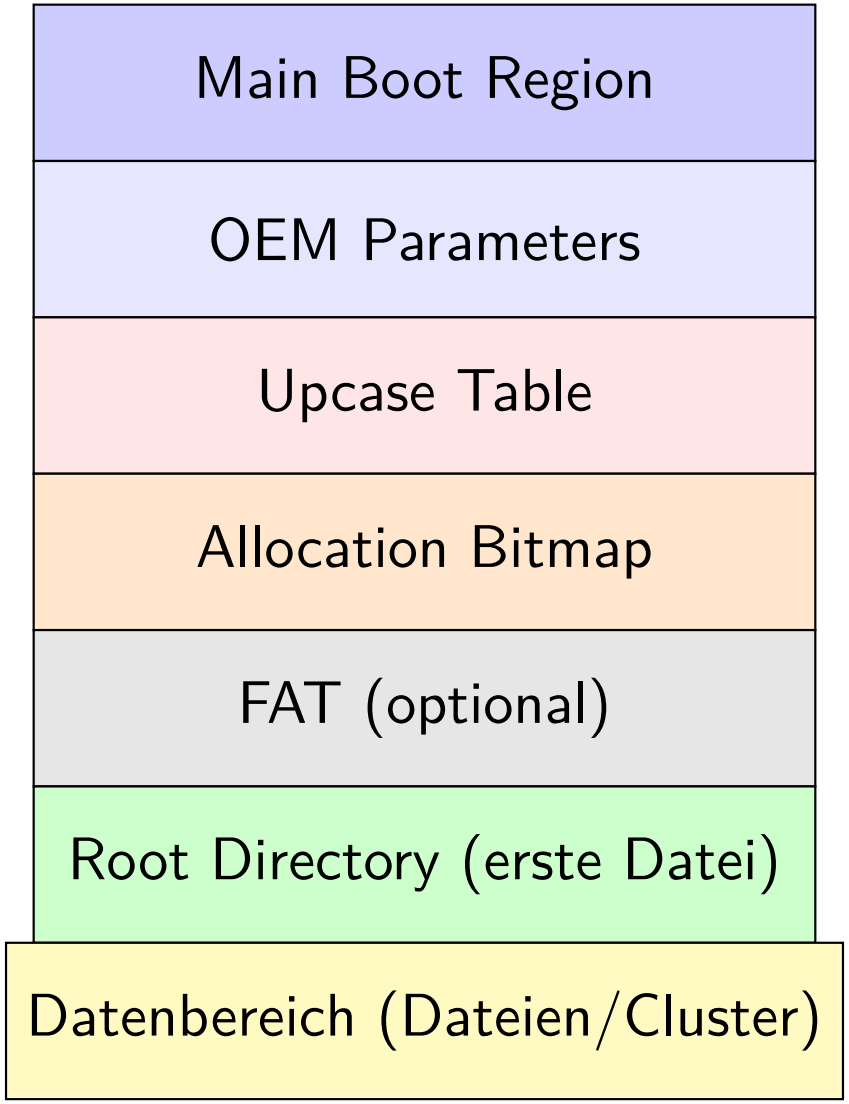
Cluster-Verwaltung: exFAT

- **Allocation Bitmap:**
  - 1 Bit pro Cluster
  - zeigt, ob Cluster frei oder belegt ist
- **FAT:**
  - optional genutzt zur Verkettung von Clustern
  - bei Fragmentierung erforderlich
- **Contiguous Flag:**
  - kennzeichnet zusammenhängende Clusterketten
  - beschleunigt sequentiellen Zugriff

Speicherlayout von FAT32



Speicherlayout von exFAT



Aufbau exFAT - Bootsektor

Offset	Länge	Inhalt
0x00	3	Sprungbefehl zum Bootcode
0x03	8	OEM-ID (z.¿B. ËXFAT ")
0x0B	53	Reserved (für zukünftige Erweiterungen)
0x40	8	Volume Offset (Startsektor des Volumes relativ zum phys. Medium)
0x48	8	FAT Offset (Start der FAT)
0x50	8	FAT Länge (Anzahl Sektoren)
0x58	8	Cluster Heap Offset (Start Datenbereich)
0x60	4	Anzahl Cluster im Datenbereich
0x64	4	Root Directory Start-Cluster
0x68	4	Volume Serial Number
0x6C	2	Datei-System-Version (z.¿B. 1.0 = 0x0100)
0x6E	2	Volume Flags
0x70	1	Bytes pro Sector (2er-Logarithmus, z.¿B. 9 = 512 Bytes)
0x71	1	Sektoren pro Cluster (2er-Logarithmus)
0x72	1	Anzahl FATs (typisch: 1)
0x73	1	Drive Select (meist 0x80)

Aufbau exFAT - Extended Boot Region (EBR)

Offset	Länge	Inhalt
0x74	1	Active FAT Index
0x75	1	Reserved
0x76	2	Bytes pro FAT-Eintrag (meist 4)
0x78	390	Boot Code
0x1FE	2	Signatur: 0xAA55 (bootfähig, wie bei FAT)

Kritik und Einschränkungen

- Kein Journaling - nicht robust gegen Abstürze
- Microsoft-Patente (bis 2019 ein Problem)
- Keine Rechteverwaltung wie bei NTFS
- Nicht geeignet für Server-Umgebungen

Fazit

- exFAT: optimale Wahl für Flash-Speicher mit großen Dateien
- Kombination aus Einfachheit und Flexibilität
- Für Wechseldatenträger meist besser als FAT32

Das Ext2-Filesystem im Überblick

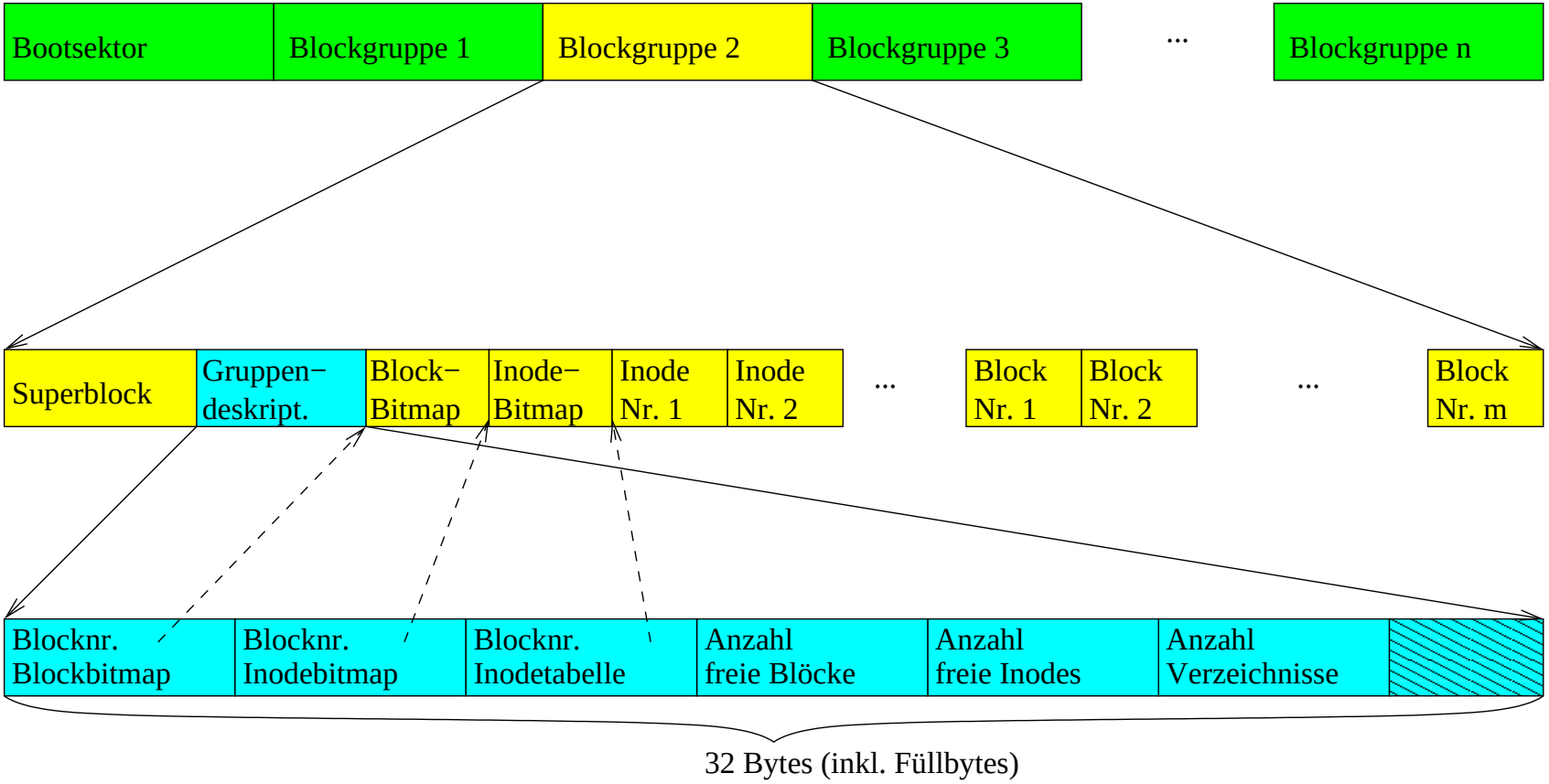
- Viele Eigenschaften vom Unix-Dateisystem (UFS) übernommen
- z.B. Blöcke (eigentlich Cluster!), Inodes und Verzeichnisse
- zahlreiche Erweiterungen: Fragmente, ACLs (Listen zur Zugriffskontrolle), Wiederherstellung gelöschter Dateien gibt es als Patches
- Versionierungssystem erlaubt Erweiterbarkeit mit Abwärtskompatibilität (genutzt von ext3, ext4)
- Einheitliche Ablage im Little Endian Format (damit plattformunabhängig)
- Einteilung in Blöcke (= Cluster) fester Größe von 1kb, 2kb oder 4kb (Festlegung bei Formatierung)

Ext2 - Eckdaten

Blockgröße	1 KB	2 KB	4 KB	8 KB (Alpha)
max. Dateigröße	16 GB	256 GB	2 TB	2 TB
max. Dateisystemgröße	2 TB	8 TB	16 TB	32 TB

- max. Anzahl von Unterverzeichnisen beträgt 32768
- In der Praxis können zahlreiche Anwendungsprogramme nicht mit Dateigrößen $>$ 2GB umgehen

Das Ext2-Filesystem im Überblick



Superblock

- Direkt nach dem Bootsektor, also nach den ersten 1024 Bytes
- Superblock enthält wichtige Verwaltungsinformation zu
  - Größenangaben zu Blöcken
  - Anzahl von freien Blöcken und Inodes
  - Anzahl von Blöcken und Inodes in Blockgruppe
  - Zähler für Mounts (für Filesystemcheck)
  - Information zu Fragmenten
  - u.a.
- Superblock zu Beginn mehrerer Blockgruppen

Aufbau des Superblock

0	Anzahl der Inodes		Anzahl der Blöcke	
8	Anzahl reservierter Blöcke		Anzahl der freien Blöcke	
16	Anzahl freier Inodes		erster Datenblock	
24	Blockgröße		Fragmentgröße	
32	Blöcke je Gruppe		Fragmente je Gruppe	
40	Inodes je Gruppe		Zeit des Mountens	
48	Zeit des letzten Schreibens		Mountzähler	max. Mounts
56	Signatur	Status	Fehlerverhalten	Füllwort
64	Zeit des letzten Checks		max. Check−Zeitintervall	
72	Betriebssystem		Revision Dateisystem	
80	RESUID	RESGID		

Rolle der Bitmaps

Block-Bitmap

- Ist genau ein Block groß
- Ein Bit pro Block in der Blockgruppe zeigt Belegung an
- Eine Blockgröße von 1024 reicht damit für 8192 Blöcke

Inode-Bitmap

- Ist ebenfalls ein Block groß
- Gibt Infos, welche Inodes frei oder belegt sind

Zusätzliche Dateiattribute in ext2

Konstante	Wert	Bemerkung
EXT2_SECRM_FL	1	sicheres Löschen
EXT2_UNRM_FL	2	Undelete (n.i.)
EXT2_COMPR_FL	4	komprimierte Datei (n.i.)
EXT2_SYNC_FL	8	synchrones Schreiben
EXT2_IMMUTABLE_FL	16	unveränderliche Datei
EXT2_APPEND_FL	32	Append-only-Datei
EXT2_NODUMP_FL	64	Datei nicht archivieren

Wichtige Infos zu Dateisystem über dumpe2fs

```
...
Inode count:              1665024
Block count:              13317852
Reserved block count:     665892
Free blocks:              8095987
Free inodes:              1411854
First block:              1
Block size:               1024
Fragment size:            1024
Blocks per group:         8192
Fragments per group:      8192
Inodes per group:         1024
Inode blocks per group:   128
Filesystem created:       Sat Oct 28 11:14:04 2006
...
```

Beispiel Blockgruppen (dumpe2fs)

```
Gruppe 0: (Blöcke 1-8192)
  Primary Superblock in 1, Gruppendeskriptoren in 2-52
  Block bitmap in 53 (+52), Inode Bitmap in 54 (+53)
  Inode-Tabelle in 55-182 (+54)
  0 freie Blöcke, 1013 freie Inodes, 2 Verzeichnisse
  Freie Blöcke:
  Freie Inodes: 12-1024
Gruppe 1: (Blöcke 8193-16384)
  Backup Superblock in 8193, Gruppendeskriptoren in 8194-8244
  Block bitmap in 8245 (+52), Inode Bitmap in 8246 (+53)
  Inode-Tabelle in 8247-8374 (+54)
  0 freie Blöcke, 1024 freie Inodes, 0 Verzeichnisse
  Freie Blöcke:
  Freie Inodes: 1025-2048
```

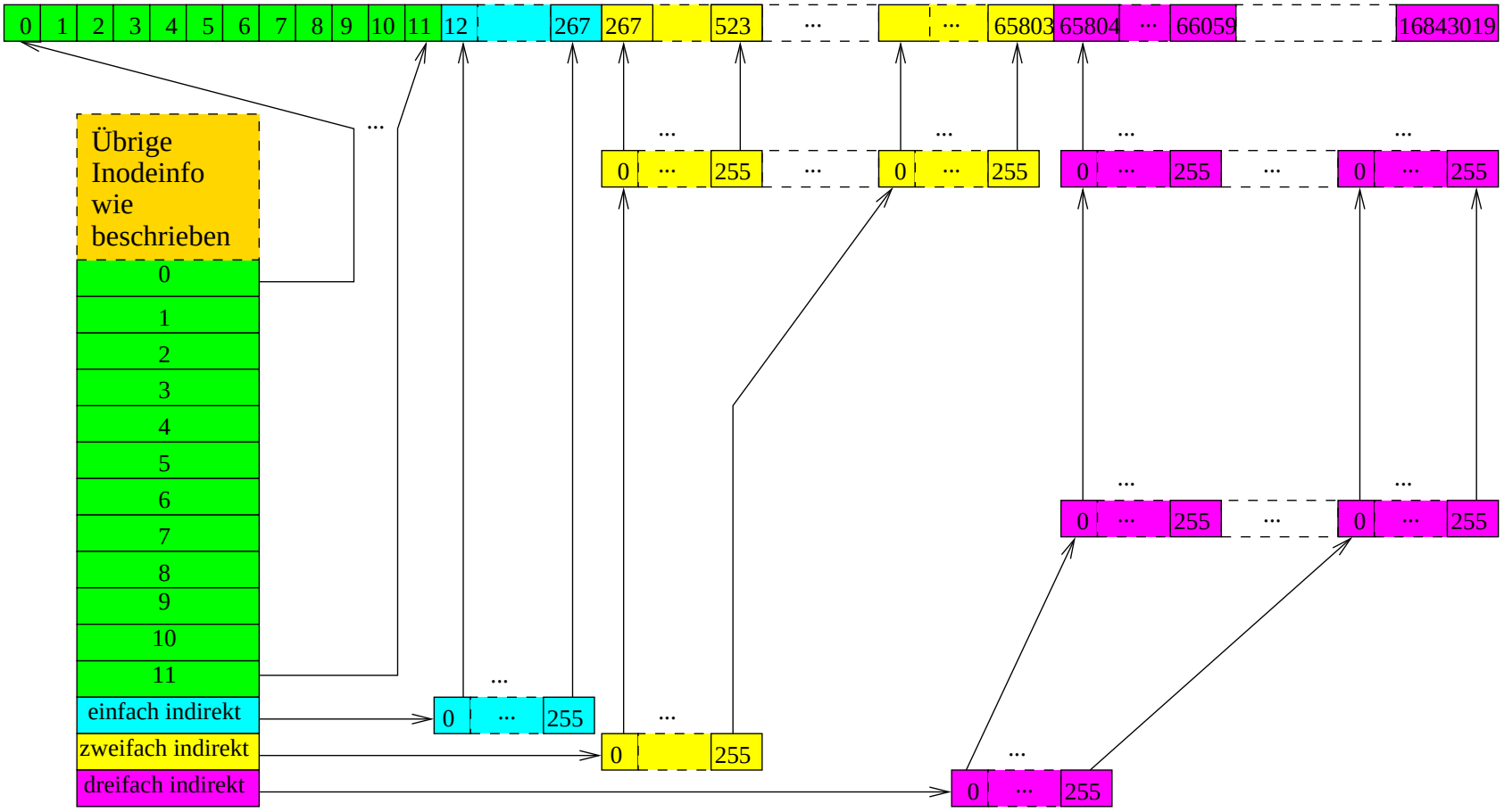
Inode-Tabelle

- Erstreckt sich über mehrere zusammenhängende logische Blöcke
- 128 Bytes für jede Inode der Blockgruppe reserviert
- Jeder Inode repräsentiert ein Objekt des Dateisystems
  - Datei
  - Verzeichnis
  - anderes wie z.B. symbolische Links, Socket, Gerät, etc.
- Metadaten je Inode sind
  - Zugriffsrechte, Besitzer, Gruppe, Flags, Größe/Anzahl Blöcke
  - Zugriffs- und Änderungszeitpunkte
  - erweiterte Attribute und ggf. ACLs
  - 12 direkte Verweise auf Datenblöcke
  - sowie einfach, zweifach und dreifach indirekte Verweise

Inhalt eines Inode

- Eigentümer der Datei
- Gruppe der Datei
- Zugriffsrechte der Datei
- Dateityp: Datei, Verzeichnis, Link, Gerät, ...
- Dateigröße in Bytes
- Anzahl der Referenzen
  - In Unix/Linux mehrere Verweise (Hardlinks) auf eine Datei möglich
  - Datei damit unter verschiedenen Namen ansprechbar
  - Löschung der Datei, wenn Zähler Wert 0 hat
- Zeitdaten: letzte Inodeänderung, Zugriff und Modifikation
- Der Dateiname selbst steht daher nicht im Inode!

Inodes und indirekte Adressierung am Beispiel ext2



Ext3 Dateisystem

- Ext3 ist abwärtskompatibel zu Ext2
- Alles was bisher zu Ext2 gesagt wurde kann auch für Ext3 verwendet werden (inkl. der Werkzeuge)
- Ext3 unterscheidet sich von Ext2 nur durch ein zusätzliches Journal
- Journal ist als zusätzliche Datei (Journaldatei) in Ext2 realisiert
- Das Journal-Log von Ext3 unterscheidet
  - Meta-Daten-Blöcke
  - Descriptor-Blöcke
  - Header-Blöcke
- Speicherplatz des Logs wird zyklisch benutzt (Ringpuffer)
- Ist der Puffer voll, werden erst alle Daten geflushed (zurückgeschrieben)

Journal ansehen (1)

Journal lässt sich mit `debugfs` betrachten, z.B. liefert

```
debugfs -R 'logdump /tmp/journal-dump.txt'
/dev/loop0
```

Ort und Art der Journalblöcke (loop0 ist ein Loopbackdevice, behandeln wir später)

```
Journal starts at block 21065, transaction 9918
Found expected sequence 9918, type 1 (descriptor block) at block 21065
Found expected sequence 9918, type 2 (commit block) at block 21070
Found expected sequence 9919, type 2 (commit block) at block 21071
Found expected sequence 9920, type 5 (revoke table) at block 21072
Found expected sequence 9920, type 1 (descriptor block) at block 21073
Found expected sequence 9920, type 2 (commit block) at block 21078
Found expected sequence 9921, type 1 (descriptor block) at block 21079
Found expected sequence 9921, type 2 (commit block) at block 21083
Found expected sequence 9922, type 2 (commit block) at block 21084
...
No magic number at block 29700: end of journal.
```

Journal ansehen (2)

Über entsprechende Optionen lassen sich auch die Dateninhalte betrachten, z.B. liefert
debugfs -R 'logdump -acs /tmp/journal-dump3.txt' /dev/loop0

```
Found expected sequence 3856, type 1 (descriptor block) at block 1
Dumping descriptor block, sequence 3856, at block 1:
FS block 6 logged at journal block 2
0000: 00000000 00000000 456c156c 456c156c ........l.lEl.lE
0010: 456c156c 00000000 00000000 00000000 l.lE............
0020: 00000000 00000000 00000000 00000000 ................
0030: 00000000 00000000 00000000 00000000 ................
0040: 00000000 00000000 00000000 00000000 ................
0050: 00000000 00000000 00000000 00000000 ................
0060: 00000000 00000000 00000000 00000000 ................
0070: 00000000 00000000 00000000 00000000 ................
0080: 03e841ed 00001000 45d03b5e 45b5ddc6 .A......^;.E...E
0090: 45b5ddc6 00000000 00030064 00000008 ...E....d.......
```

Ext4 Dateisystem

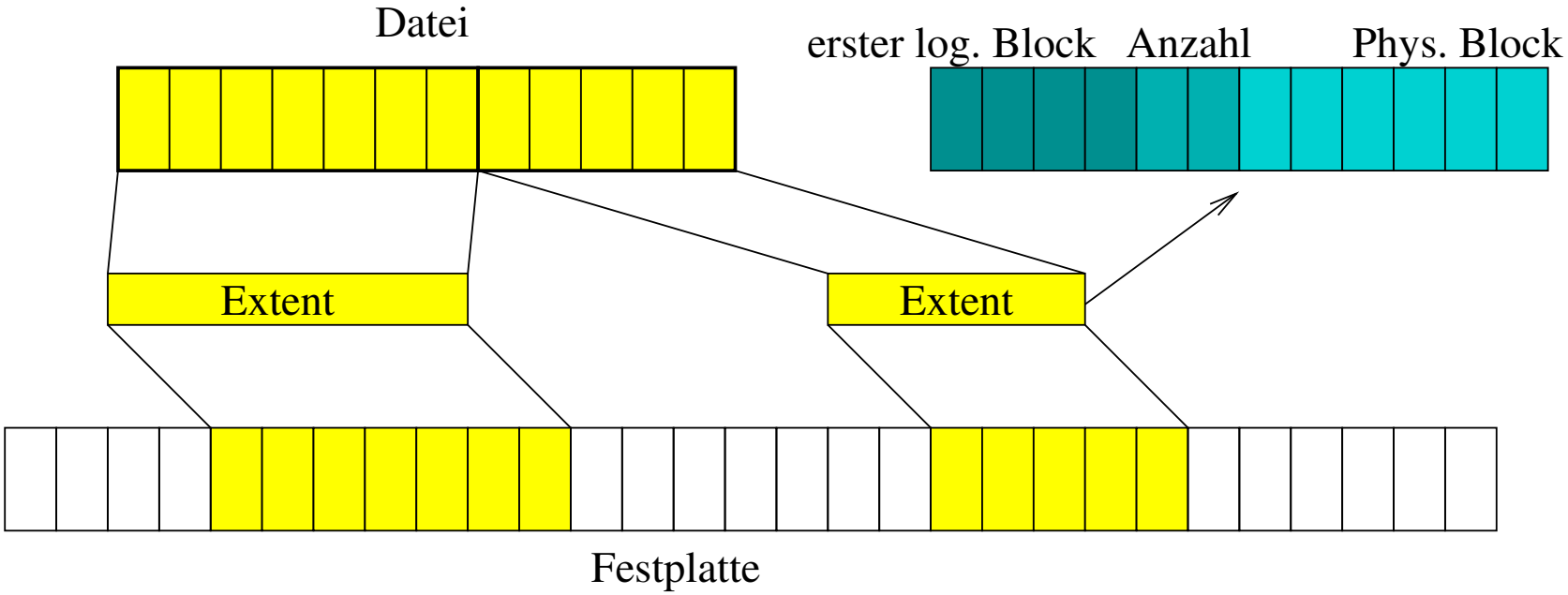
- Einführungsgrund: 16TB-Beschränkung bei 4k-Blockgrößen aufgeben
- Standardblockgröße weiterhin 4kB
- Erweiterung von 32-Bit Blocknummern auf 48 Bit
- Neue Interpretation der Blocknr. richtet sich nicht nach 512-Byte-Blöcken aus, sondern an Blockgröße des Dateisystems (i.d.R. 4K)
- Aus Kompatibilitätsgründen Flag für alte/neue Interpretation
- Für 48-Bit-Blocknr. Nutzung zweier bislang bei ext2/3 ungenutzte Bytes im Inode
- Statt indirekter Blockadressierung werden Extents verwendet
- Neue Grenze: 1024 PByte

Datenstruktur zu Entents in Ext4

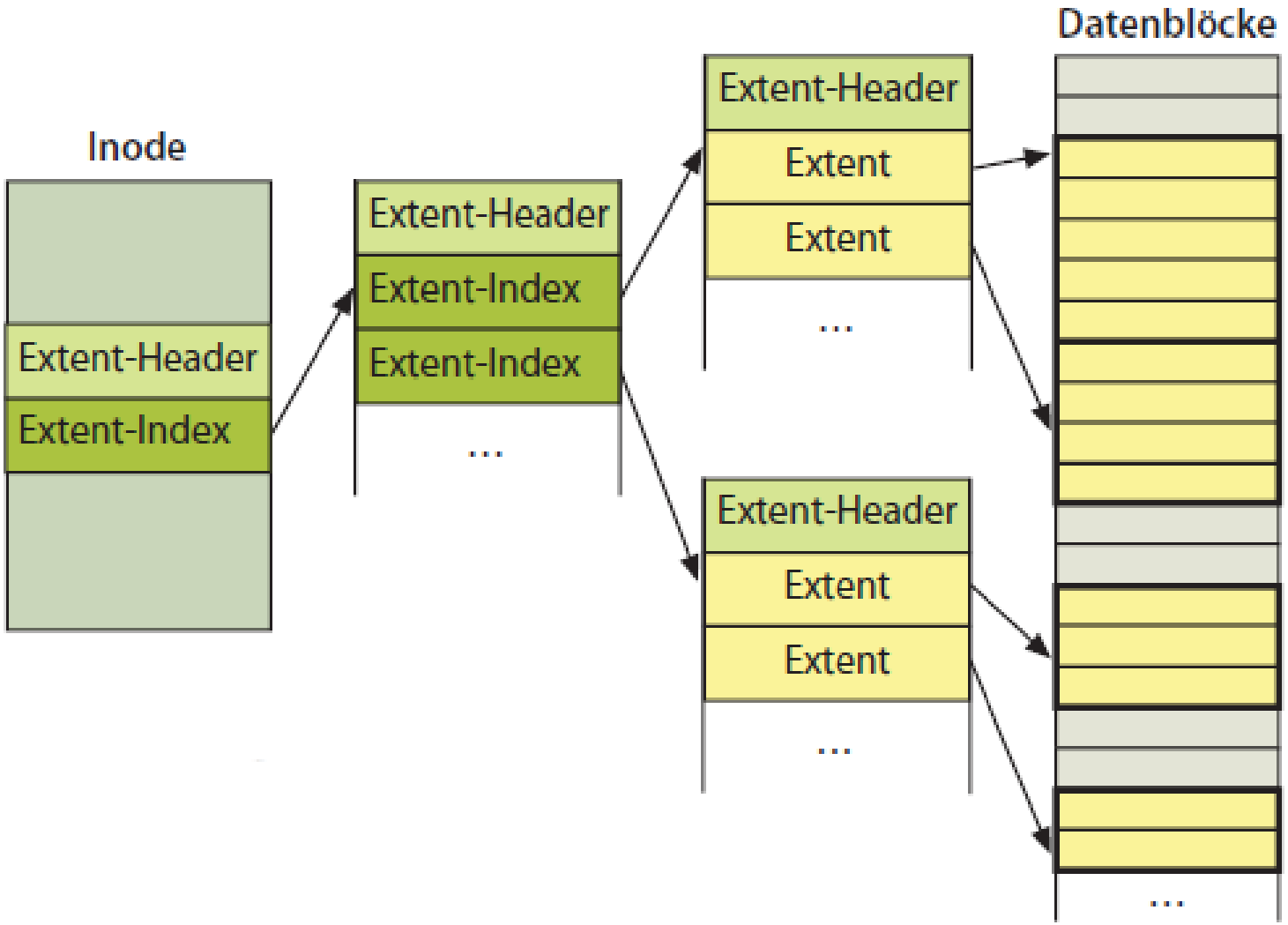
```
struct ext3_extent {
    /* first logical block extent covers */
    __u32   ee_block;
    /* number of blocks covered by extent */
    __u16   ee_len;
    /* high 16 bits of physical block */
    __u16   ee_start_hi;
    /* low 32 bits of physical block */
    __u32   ee_start;
};
```

- Extents adressieren (möglichst großen) Blockbereich
- Da 32-bittige Blockanzahl, maximal 2^{32} Blöcke pro Datei
- Problem: Stark fragmentierte Daten

Ext-4 Dateisystem - Extents



Extents-Baum



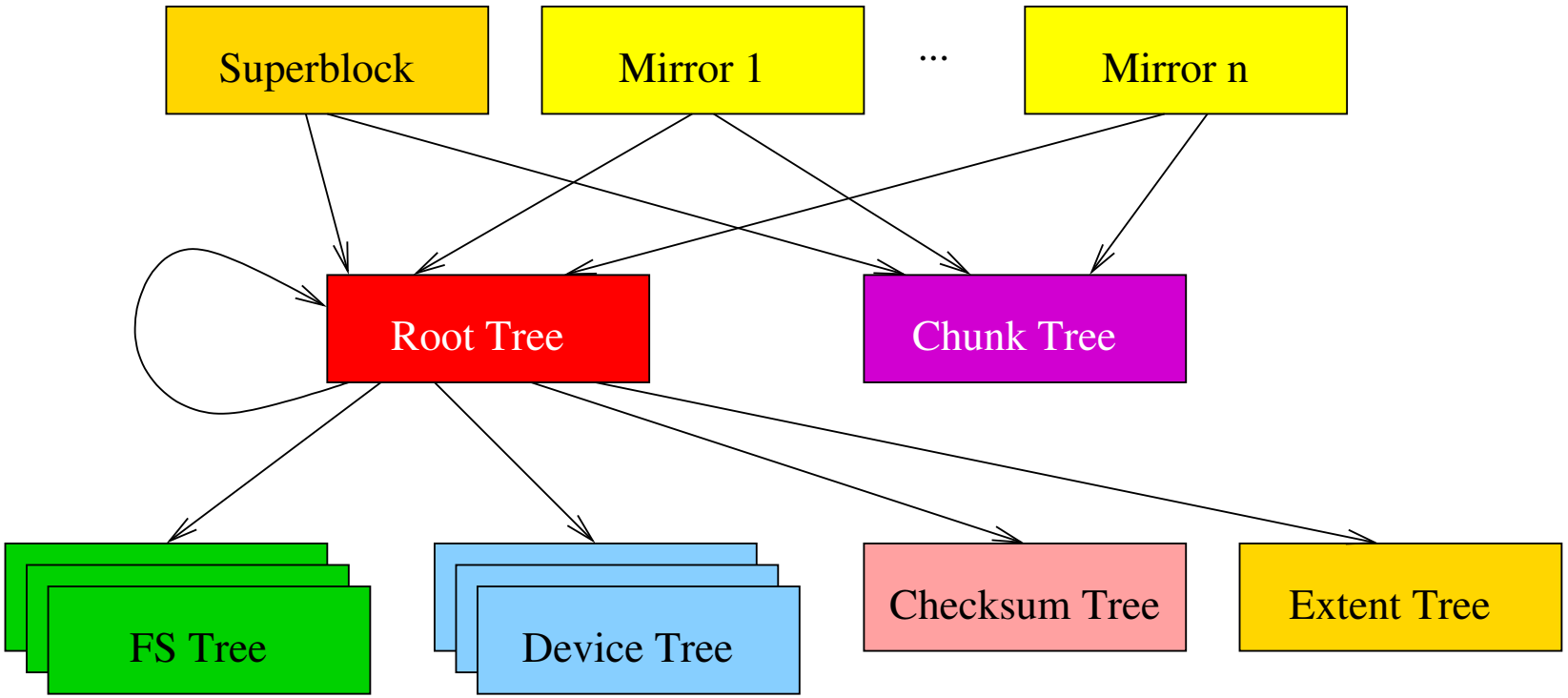
Btrfs - Eigenschaften

- File System Tree: repräsentiert ein Subvolume
- enthält alle Metadaten von Dateien und Verzeichnissen.
- Extent Tree: indiziert den allokierten Speicher
- Jeder Extent enthält eine Referenz auf die Datei oder das Blatt, zu dem die Daten gehören.
- Chunk Tree: teilt alle Datenträger (Block-Gruppen) ein, bildet logische Chunks auf physikalische Blöcke ab.
- Device Tree hält für jeden Datenträger fest, welche Blöcke welchen Chunks zugewiesen sind.
- Checksum Tree: Liste mit Prüfsummen für alle beinhaltenden Blöcke jedes Extent
- Prüfsummen für Metadaten und eingebettete Dateien direkt in den Blättern des FS

Btrfs - Geplante Eigenschaften

- Btrfs wird noch ständig weiter entwickelt
- Daher zahlreiche Eigenschaften geplant:
  - höhere RAID-Level
  - Verschlüsselung
  - Online Dateisystemprüfung
  - weitere Verbesserungen Snapshothandling
- Btrf kennt nur einen einzigen generischen Baum mit elementaren Datenstrukturen
- Jeder Knoten und jedes Blatt beginnt mit einem Block Header

Übersicht über die Strukturen eines Btrfs



Btrfs - Weitere Bäume

- Log Tree: Beeinhaltet Änderungen die noch nicht geschrieben wurden.
- Quota Tree: Enhält Konfigurationen für Speicherkontingente, welcher hat wieviel Platz
- Reloc Tree: Infos, die beim Verschieben von Extents benötigt werden (z.B. für Defragmentierung)
  - Eindeutige ID für jeden Baum
  - Bäume mit nur einer Instanz: Feste ID < 265
  - Root Tree hat ID 1 mit Selbstreferenz
- Dateiänderung $\longrightarrow$ Änderung der Metadaten in den Blättern
- Änderungen breiten sich durch Copy-On-Write nach oben aus $\longrightarrow$ neue Wurzel

Btrfs - Checkpoints (1)

- Änderungen an Extents, Prüfsummen etc. pflanzen sich also fort bis neue Wurzel entsteht
- Verweise auf neuen Wurzeln landen im Root Tree
- Neue Wurzel die dadurch entsteht landet im Superblock = neuer Checkpoint
- Stürzt Filesystem oder BS während des Vorgangs ab wird der alte Root Tree weiterverwendet.
- Performanceprobleme: Damit diese nicht auftreten, Änderungen zunächst im Arbeitsspeicher
- Ab einer gewissen Mindestdatenmenge, nach spätestens 30sec. $\longrightarrow$ Persistenz
- Dirty Blocks im Cache und „Late Allocation“
- Checkpoint ID: Aufsteigende Transaktionsnummer

Btrfs - Checkpoints (2)

- Rolling Checkpoints: Letzte vier Verweise im Root Tree
- Daher Rücksprung von max. 2 Minuten im Dateisystem
- Checkpoints: Reservierung eines gesonderen Bereiches auf Datenträger
- Vorteil: Es stehen immer freie Blöcke zur Verfügung
- Lokalität bei Blättern
- Änderungen am Superblock kein Copy on Write (COW)

Datenstrukturen (1): Block Header beim Btrfs

```
struct btrfs_header {
    u8 csum[BTRFS_CSUM_SIZE];
    u8 fsid[BTRFS_FSID_SIZE];
    __le64 bytenr;
    __le64 flags;
    u8 chunk_tree_uuid[BTRFS_UUID_SIZE];
    __le64 generation;
    __le64 owner;
    __le32 nritems;
    u8 level;
}
```

Btrfs - Block Header

- Enthält Informationen über den Block, in dem sich ein Knoten oder Blatt befindet
- csum enthält die Prüfsumme des blocks
- fsid entspricht der eindeutigen UUID des Dateisystems
- bytenr ist der logische Offset des Blockbeginns
- flags geben an, ob der Block bei einem Checkpoint mittels COW noch geschrieben werden muß
- generation Nummer des Checpoints der Erstellung des Blocks
- In den Blättern: Nach dem Blockheader ein Array mit Items

Datenstrukturen (2)

Blatt (Leaf) beim Btrfs:

```
struct btrfs_leaf {
    struct btrfs_header header;
    struct btrfs_item items[];
}
```

Ein Item beim Btrfs:

```
struct btrfs_item {
    struct btrfs_disk_key key;
    __le32 offset;
}
```

Datenstrukturen (3)

Btrfs Key

```
struct btrfs_disk_key {
    __le64 objectid;
    u8 type;
    __le64 offset;
}
```

Btrfs Node

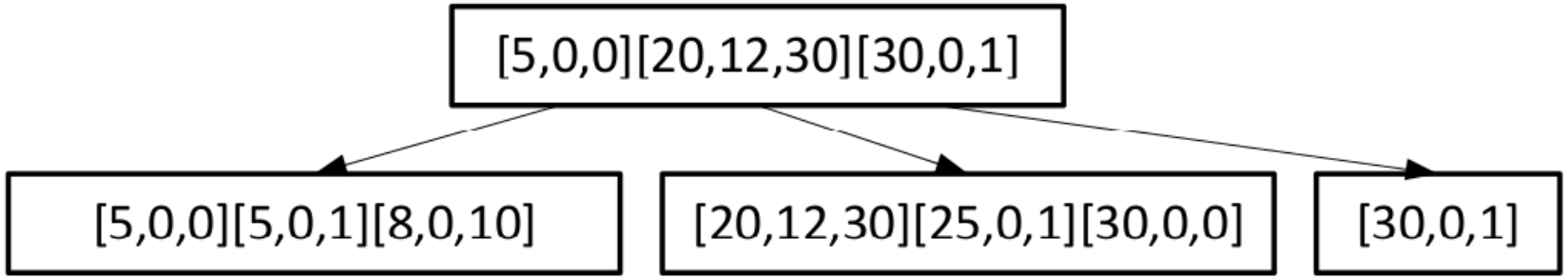
```
struct btrfs_node {
    struct btrfs_header header;
    struct btrfs_key_ptr ptrs[];
}
```

Datenstrukturen (4)

Btrfs Key

```
struct btrfs_key_ptr {
    struct btrfs_disk_key key;
        __le64 blockptr;
        __le64 generation;
}
```

Struktur innerer Knoten



Datenstrukturen (5): Btfs Inode

```
struct btrfs_inode_item {
        __le64 generation;
        __le64 transid;
        __le64 size, nbytes;
        __le64 block_group;
        __le32 nlink;
        __le32 uid, gid, mode;
        __le64 rdev;
        __le64 flags;
        __le64 sequence;
        __le64 reserved[4];
struct btrfs_timespec atime, ctime;
struct btrfs_timespec mtime, otime;
}
```

Typen der Directory-Items

Typ	Wert
Unknown	0
File	1
Directory	2
Character Device	3
Block Device	4
FIFO	5
Socket	6
Symlink	7
Extended Attribute (xattr)	8

Typen der Block-Group-Items

Typ	Wert
Data	1
System	2
Metadata	4
Striped (RAID-0)	8
Mirrored (RAID-1)	16
Duplicated (DUP)	32
Striped and mirrored (RAID-10)	64

Btrfs Debugging - Inhalt Root Tree

Den Inhalt des Root Tree erhält man mit btrfs-debug-tree -r

```
root/# btrfs-debug-tree -r /dev/sda
root tree: 87879680 level 1
chunk tree: 20975616 level 0
extent tree key (EXTENT_TREE ROOT_ITEM 0) 87883776 level 1
device tree key (DEV_TREE ROOT_ITEM 0) 87674880 level 0
fs tree key (FS_TREE ROOT_ITEM 0) 87584768 level 0
checksum tree key (CSUM_TREE ROOT_ITEM 0) 87900160 level 1
file tree key (256 ROOT_ITEM 0) 87678976 level 0
file tree key (258 ROOT_ITEM 0) 87605248 level 0
data reloc tree key (DATA_RELOC_TREE ROOT_ITEM 0) 29380608 level 0
total bytes 1164967936
bytes used 115666944
uuid f32310af-209f-43f7-b868-258f89ab309f
Btrfs v0.20-rc1-37
```

Btrfs Debugging - Inhalt einer Block Adresse

Inhalt einer gegebenen Blockadresse mit btrfs-debug-tree -b

```
root:/# btrfs-debug-tree -b 87678976 /dev/sda
leaf 87678976 items 12 free space 2885 generation 9 owner 256
fs uuid f32310af-209f-43f7-b868-258f89ab309f
chunk uuid ff267765-3f5d-477f-b3e5-6b90072b1769
item 0 key (256 INODE_ITEM 0) itemoff 3835 itemsize 160
inode generation 5 transid 8 size 48 ...
item 1 key (256 INODE_REF 256) itemoff 3823 itemsize 12
inode ref index 0 namelen 2 name: ..
item 2 key (256 DIR_ITEM 785575738) itemoff 3779 itemsize 44
location key (258 INODE_ITEM 0) type 1
namelen 14 datalen 0 name: randomFile_100
...
item 11 key (258 EXTENT_DATA 0) itemoff 3185 itemsize 53
extent data disk byte 330956800 nr 104857600
extent data offset 0 nr 104857600 ram 104857600
extent compression 0
```

Positionen Superblock mit Kopien im Btrfs

Typ	Offset (hex)	Offset (byte)
Primary	0x1 0000	64KiB
Mirror	0x400 0000	64MiB
Mirror	0x40 0000 0000	256GiB
Mirror	0x4 0000 0000 0000	1PiB

Reiser FS - Eigenschaften

- ReiserFS, Version 1-2 (letztere versehentlich entstanden) basiert auf $B^\star$-Bäumen
- Ab Version 3 Journaling für Metainformationen (Verwaltungsstrukturen) ergänzt
- Mit Kernel 2.6 auf „echtes“ Journaling erweitert (Option `data=journal`)
- Reiser4 ist eine vollständige Neuentwicklung
  - Verwendung von „Dancing Trees“
  - d.h. unzureichend gefüllte Knoten werden nur verschmolzen, wenn durch Speicherknappheit gefordert
  - Daten werden für Journaling nicht mehr 2x geschrieben
  - Daten werden an Zielposition geschrieben, Journal bis zum Commit per Referenz darüber gelegt
  - Plugin-Struktur für z.B. Verschlüsselung und Komprimierung

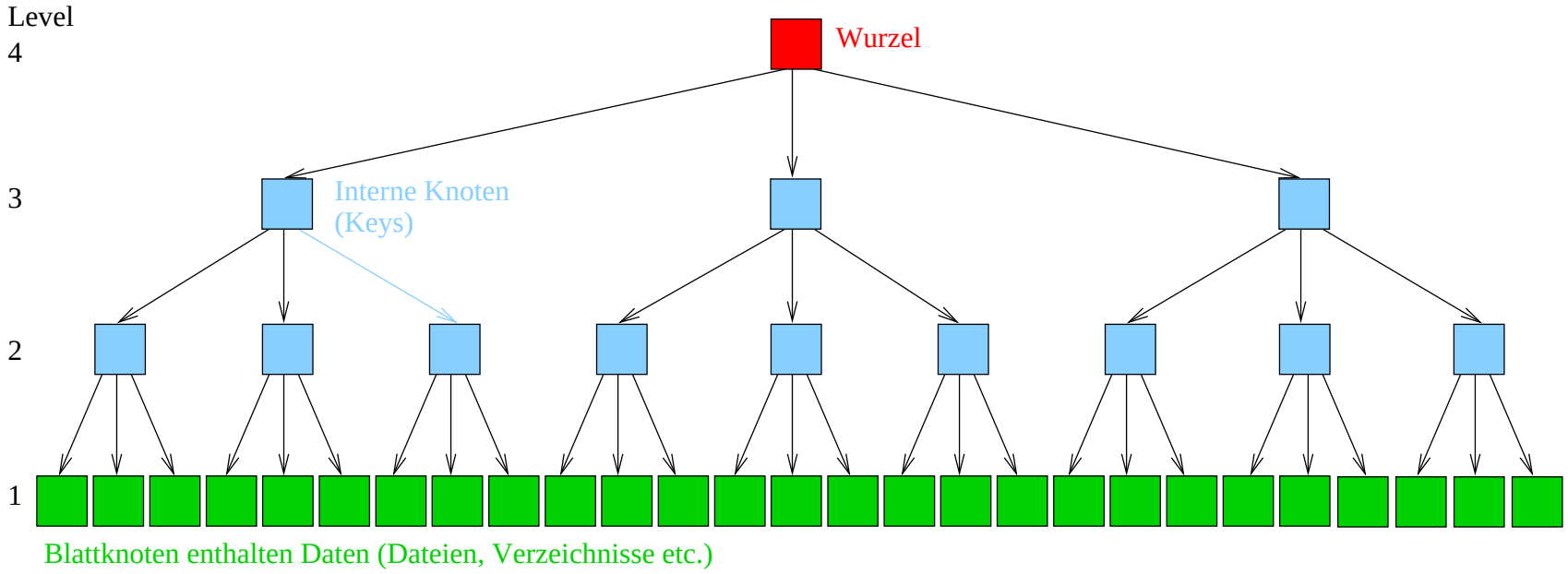
Reiser FS - Eckdaten der letzten 3er Versionen

	3.5	**3.6**
max. Anzahl Dateien	$2^{32} - 3$	$2^{32} - 3$
max. Anzahl Dateien in Verzeichnis	518701895 (in der Praxis nur ca. 1.200.000 wg. Hashfunktion r5)	$2^{32} - 4$ (in der Praxis nur ca. 1.200.000 wg. Hashfunktion r5)
max. Größe einer Datei	$2^{31} - 1$ (ca. 2G)	$2^{60} - 1$ (ca. 1E, aber auf Rechnern mit 32-Bit-int auf "nur" 8T limitiert
max. Anzahl Hardlinks	2^{16}	2^{32}
max. Größe Dateisystem	2^{32} 4k-Blöcke (=16T)	2^{32} 4k-Blöcke (=16T)

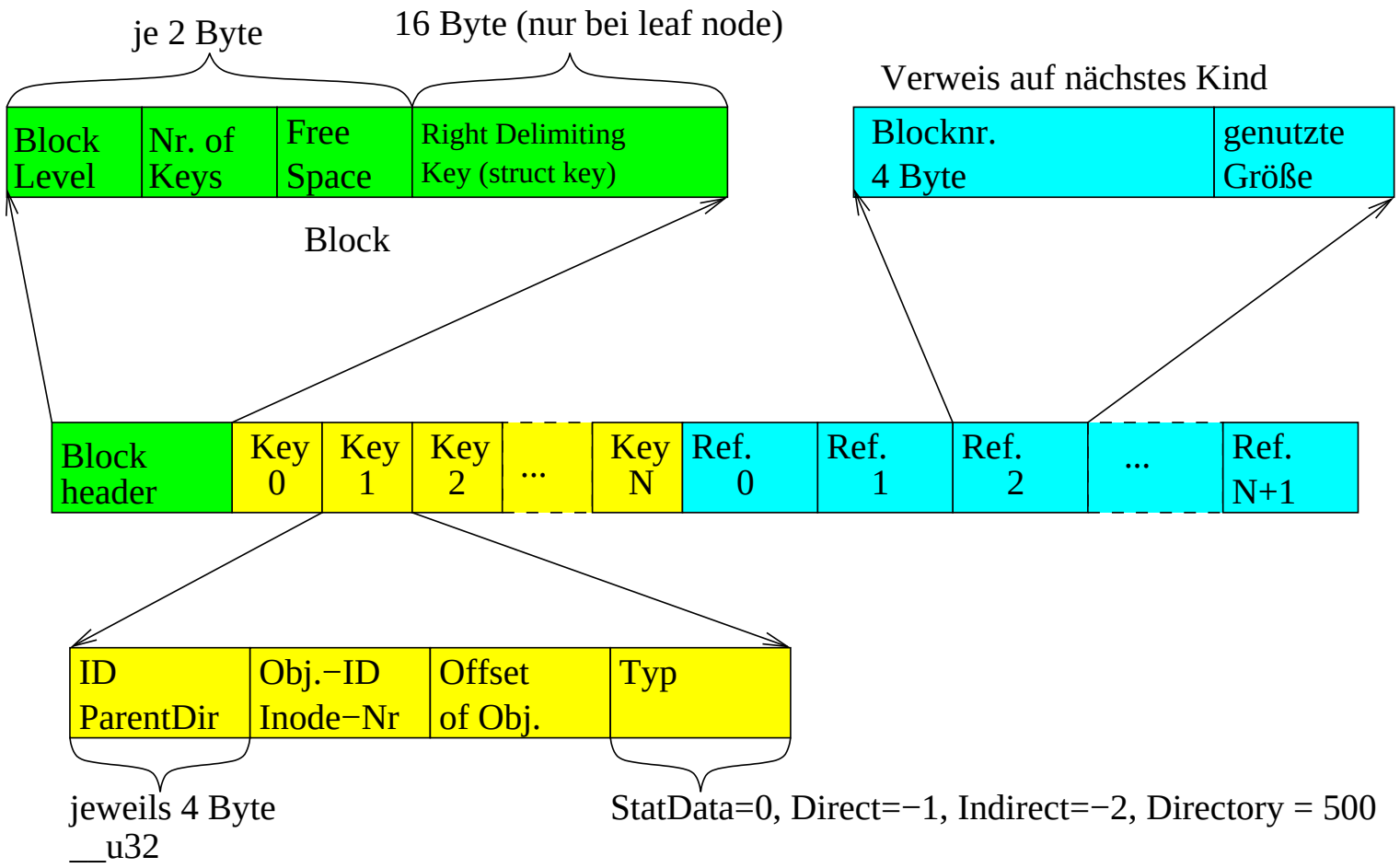
Reiser FS - Wichtige Werkzeuge und Optionen

- Optionen beim Mounten:
  - `notail` deaktiviert Zusammenfassung kleiner Dateien in Inodes
  - `nolog` deaktiviert Journaling
- `reiserfstune` für unsere Zwecke weniger interessant
- `reiserfsck` erlaubt nicht nur Überprüfung des Dateisystems, sondern kann z.B. auch die Bäume bei Fehlern neu aufbauen
- `debugreiserfs` ist ein wichtiges Analysetool, das zur Informationsgewinnung hilfreich sein kann.

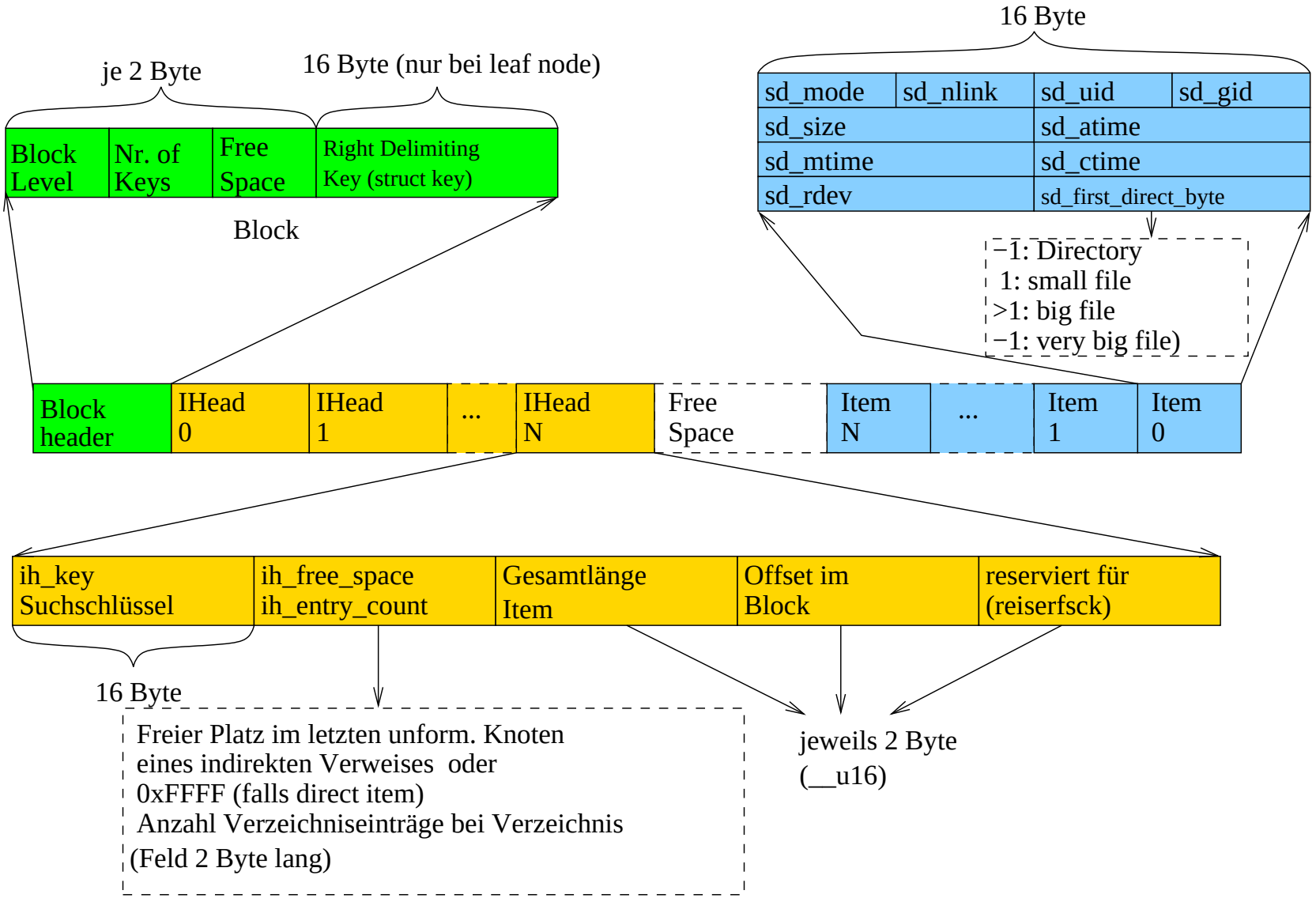
Grundidee von ReiserFS



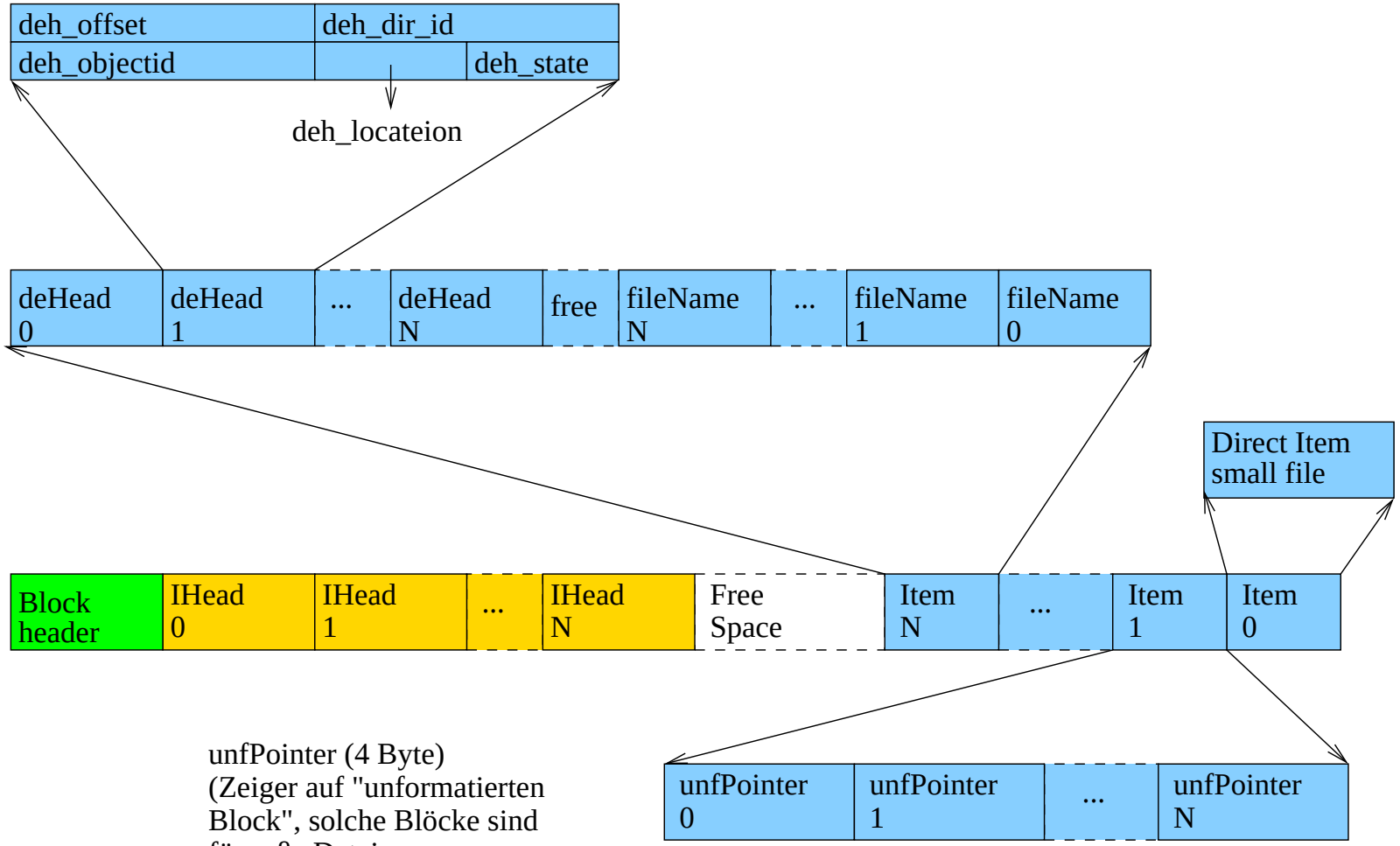
Struktur eines internen Knoten



Struktur eines Blattes (1)



Struktur eines Blattes (2)



Wichtige Infos erhält man mit debugreiserfs

```
 Reiserfs super block in block 16 on 0xfe01 of format 3.6 with standard journal
Count of blocks on the device: 5242880
Number of bitmaps: 160
Blocksize: 4096
Free blocks (count of blocks - used [journal, bitmaps, data, reserved] blocks): 196908
Root block: 3867195
Filesystem is NOT clean
Tree height: 5
Hash function used to sort names: "r5"
Objectid map size 972, max 972
Journal parameters:
        Device [0x0]
        Magic [0x2150e5cc]
        Size 8193 blocks (including 1 for journal header) (first block 18)
        Max transaction length 1024 blocks
        Max batch size 900 blocks
        Max commit age 30
Blocks reserved by journal: 0
Fs state field: 0x0:
sb_version: 2
inode generation number: 5278686
UUID: f4929d3e-8374-4af3-80e7-95a0f2a08be2
LABEL:
Set flags in SB:
        ATTRIBUTES CLEAN
```

Wichtige Infos erhält man mit debugreiserfs

Mit der Option -J erhält man zusätzlich den Header des Journals

```
... wie letzte Folie ...
Journal header (block #8210 of /dev/mapper/system-home):
    j_last_flush_trans_id 2021897
    j_first_unflushed_offset 7633
    j_mount_id 47291884896680
    Device [0x0]
    Magic [0x2150e5cc]
    Size 8193 blocks (including 1 for journal header)
                                        (first block 18)
    Max transaction length 1024 blocks
    Max batch size 900 blocks
    Max commit age 30
```

Ausgabe eines Internen Knotens

Mit der Option -d erhält man Infos über die Baumknoten

```
INTERNAL NODE (1763517) contains level=4, nr_items=3,
  free_space=3992 rdkey
  PTR 0: [dc_number=745833, dc_size=3536]
  KEY 0: [268 492951 0xd9 DRCT (2)]
  PTR 1: [dc_number=745832, dc_size=3896]
  KEY 1: [381 497182 0x1 DRCT (2)]
  PTR 2: [dc_number=176 3516, dc_size=3800]
  KEY 2: [91704 93722 0x0 SD (0)]
  PTR 3: [dc_number=346356, dc_size=2168]
```

Ausgabe eines Blattes

Option -d liefert auch die Inhalte der Blätter

```
=====================================================================
LEAF NODE (32797) contains level=1, nr_items=5, free_space=0 rdkey (real items 5)
-------------------------------------------------------------------------------
|###|type|ilen|f/sp| loc|fmt|fsck|                    key                      |
|   |    |    |e/cn|    |   |need|                                             |
-------------------------------------------------------------------------------
|  0|2 210 0x1efae180 DIR (3), len 224, location 3872 entry count 8, fsck need 0, fo
rmat old|
###: Name                      length    Object key           Hash     Gen number
  0: "prologbook.ps.gz.part    "( 21)             [2 214]   519758208    0, loc 200,
 state 4 "r5"
  1: "PROLOG11.pdf             "( 12)             [2 215]   543282304    0, loc 184,
 state 4 "r5"
  2: "sort.pl                  "(  7)             [2 216]   818400384    0, loc 176,
 state 4 "r5"
  3: "nine.pl                  "(  7)             [2 217]  1203356544    0, loc 168,
 state 4 "r5"
...
  6: "nnine.pl                 "(  8)             [2 220]  1804018304    0, loc 144,
 state 4 "r5"
  7: "vierfarben.pl            "( 13)             [2 221]  1937955200    0, loc 128,
 state 4 "r5"
-------------------------------------------------------------------------------
|  1|2 211 0x0 SD (0), len 44, location 3828 entry count 65535, fsck need 0, format
new|
```

Undelete bei Reiserfs?

- Leider gibt es keinen `undelete`-Befehl bei reiserfs!
- Dennoch lassen sich Verzeichnisse und Dateien oft mit folgendem Verfahren wiederherstellen:
  1. Imagedatei `image.dd` der Partition erstellen
  2. Mit `losetup /dev/loop0 image.dd` unter Linux ein Loopbackdevice einrichten
  3. Auf Device `/dev/loop0` anschliessend den Baum neu aufbauen lassen mit
     `reiserfsck -rebuild-tree -S -l recovery.log /dev/loop0`
  4. Anschliessend mit
     `mount /dev/loop0 /mnt/reiserdisk`
     das Ergebnis untersuchen
- Der Parameter `-S` sorgt dafür, dass auch unbenutze Bereiche nach möglichen Blöcken gescannt werden

Ausgabe von reiserfsck -rebuild-tree

```
The whole partition (256000 blocks) is to be scanned
Skipping 8218 blocks (super block, journal, bitmaps) 247782 blocks will be read
...
        Read blocks (but not data blocks) 247782
                Leaves among those 1503
                Objectids found 13031
Pass 1 (will try to insert 1503 leaves):
...
        1503 leaves read
                1496 inserted
                7 not inserted
        non-unique pointers in indirect items (zeroed) 14
Pass 2:
...
        Leaves inserted item by item 7
Pass 3 (semantic):
...
        Files found: 10806
        Directories found: 725
Pass 3a (looking for lost dir/files):
...
        Objects without names 30
        Dirs linked to /lost+found: 2
        Files linked to /lost+found 28
Pass 4 - finished       done 0, 0 /sec
        Deleted unreachable items 1
..
```

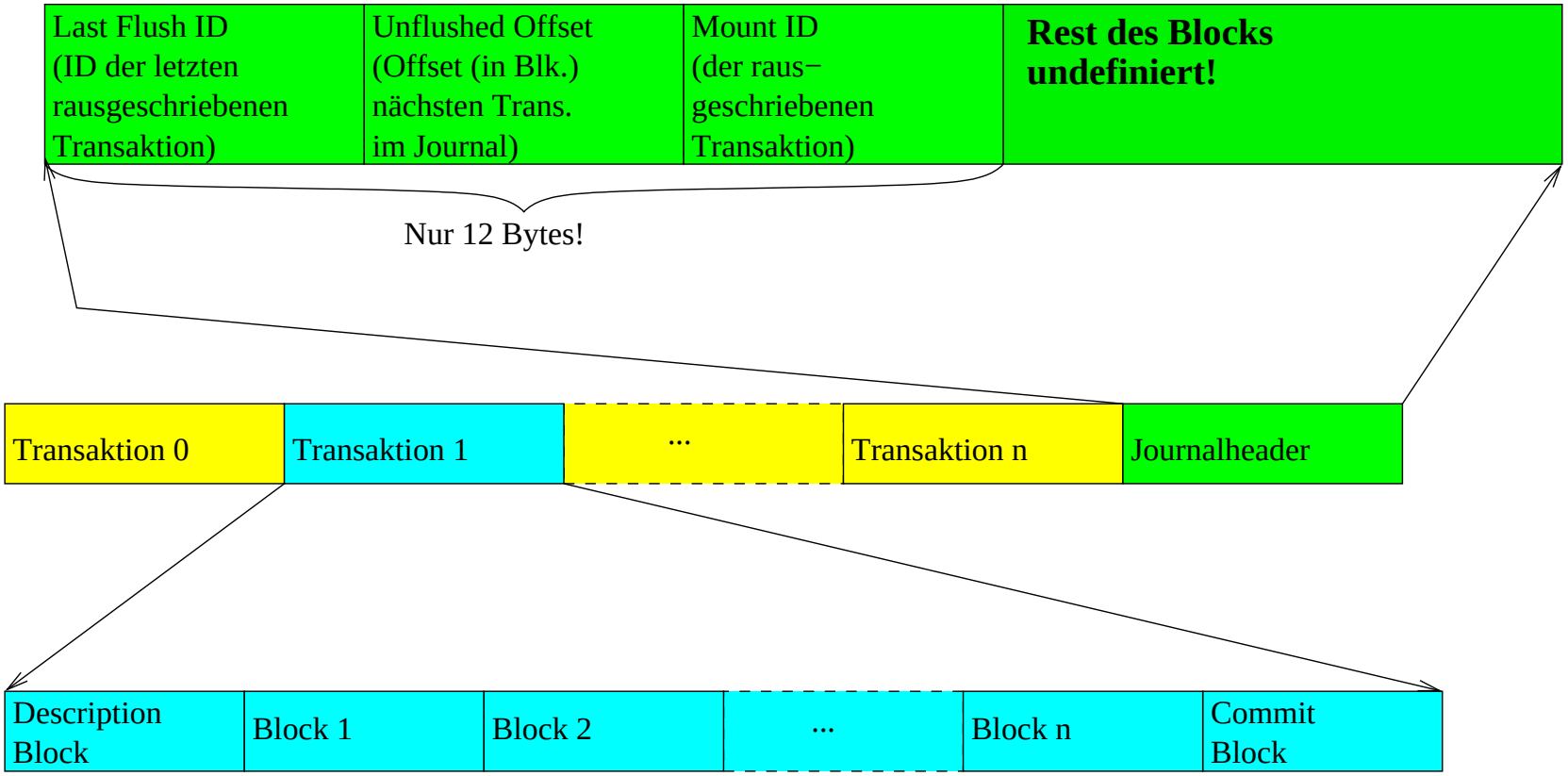
Realisierung des Journals in Reiserfs (1)

- Fortlaufende Sequenz von Festplattenblöcken
- Journal hat feste Größe (z.Zt. 8192 Blöcke für Transaktionen + 1 für Journalheader)
- Nutzung des Journals:
  - Jede Modifikation des Dateisystems wirkt zunächst nur aufs Journal
  - Zusammengehörige Transaktionen (zusammen atomar!) werden dazu ins Journal geschrieben
  - Zu späterem Zeitpunkt: Rausschreiben der Änderungen aufs Dateisystem, als erledigt markieren
- Aufbau Journal: Liste von Transaktionen variabler Länge + Journalheader
- Eine Transaktion belegt mindestens drei Blöcke
- Journal als Ringpuffer organisiert

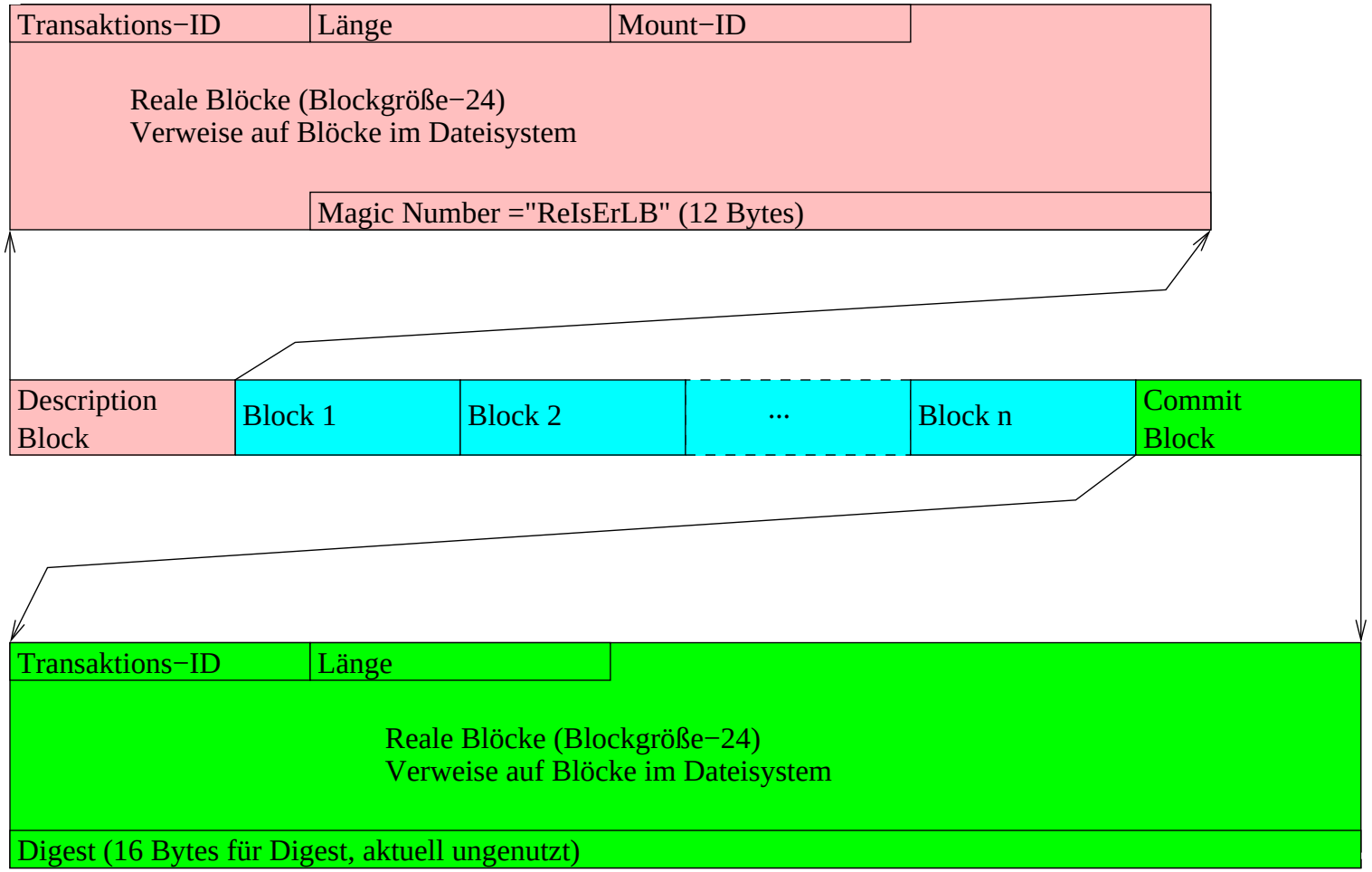
Realisierung des Journals in Reiserfs (2)

- Oft zu lesen: reiserfs3 zeichnet nur Metadaten im Journal auf, das ist nicht vollständig korrekt
- Das beschreibt lediglich den Zweck, die Integrität der Metadaten zu erhalten!
- Tatsächlich tauchen vollständige Plattenblöcke im Journal auf, die in dieser Form ins Dateisystem eingebunden werden, wenn der commit erfolgt ist.
- Zur Erinnerung: Verzeichnisse, stat. Data und kleine Dateien sind direkt in den Blattknoten gespeichert
- Daher taucht einiges davon auch im Journal auf
  $\rightarrow$ kann dazu genutzt werden frühere Datei- oder Verzeichnisversionen zu rekonstruieren!

Aufbau des Journals



Aufbau des Description- und Commitblock



Bad Block List

- Bad Block List kennzeichnet defekte Blöcke
- `reiserfstune -B dateiname` markiert die in `dateiname` enthaltenen Blöcke als defekt
- Anzeige bzw. Export in `dateiname` unter Reiserfs mit `debugreiserfs -B dateiname`
- Kann gut als „Slack Space“ verwendet werden
- Es gibt auf diversen Seiten im Internet auch fertige Dateisystemtreiber, die z.B. bad blocks zum verstecken von Daten nutzen
- Daher vermeintlich defekten Blöcken erhöhte Aufmerksamkeit widmen
- Analoges gilt auch für die anderen behandelten Dateisysteme wie z.B. ext3

Eigenschaften von NTFS

- Seit Windows NT Standardateisystem für Windows
- Löst das alte FAT ab
- Ständige Weiterentwicklung, aktuelle Version: 3.1
- NTFS-Konzeption spiegelt Windows-Semantik wider
- Daher anders als POSIX, trotzdem viele Ähnlichkeiten zu modernen Linux-/Unixdateisystemen
- z.B. ist es ein Journaled Filesystem
- z.B. redundante Speicherung wichtiger Metainformation
- im Gegensatz zu FAT hat NTFS Benutzer und Zugriffskontrolllisten (ACLs)

Größen für Blöcke und Dateien bei NTFS

- Unter Windows wird mit Cluster ein logischer Block bezeichnet.
- Cluster ist die elementare Allokierungseinheit
- Je nach Volume zwischen 512 Byte und 64 kByte groß
- Standard: 4KByte Blockgröße
- 64-Bit-Dateisystem, bisher aber nur untere 32 Bit verwendet
- Limit für Dateisystem derzeit bei 256 Terabyte
- max. Dateigröße rund 16 Terabyte
- max. $2^{32} - 1$ Dateien
- "extend-basierte" Adressierung von Dateien, in Windows als „Cluster-Run" bezeichnet

Die Master File Table (MFT)

- Die Master File Table ist das Herzstück des NTFS
- Geordnete Liste von Einträgen für sämtliche Dateien des Filesystems
- Vergleichbar mit der Inode Allocation Map von Unix-FS, aber mehr Informationen enthalten
- Jeder Dateieintrag (File Record) ist 1KByte groß (unabhängig von Clustergröße)
- Theoretisch andere Größe möglich (Wert im Bootsektor)
- Hard- und Softlinks komplizierter als bei Unix-FS
- „Alles ist eine Datei“ konsequenter als bei Unix
- MFT listet alle Metadatenstrukturen als Dateien (Dateiname beginnt immer mit $)
- MFT hat selbstbezüglichen Eintrag (Dateiname $MFT) mit Nummer 0

Metadaten der MFT

Eintrag	Bedeutung
$MFT	Die MFT selbst
$MFTMirr	Erste vier Einträge der MFT (redundant)
$LogFile	Journal für Metadatentransaktionen
$Volume	Volumeinformationen: Label, Vol-ID, Version etc.
$AttrDef	Attributinfos: IDs, Name, Größe etc.
⟩⟩.⟨⟨	Root-Directory des NTFS
$Bitmap	Allocation-Status jedes Clusters
$Boot	Bootsector und Bootcode des Filesystems
$BadClus	Cluster, die schlechte Sektoren aufweisen
$Secure	Infos zu Dateisicherheit und - zugriffskontrolle
$UpCase	Uppercase-Version jedes Unicodezeichens
$Extend	Verzeichnis, für optionale künftige Erweiterungen

Einhängen eines NTFS-Dateisystems

Eigenschaften Superblock

- Erst wird der Startcluster der MFT im Volume bestimmt
- Information im ersten Sektor dem 8kByte großen Bootsektor bzw. Superblock des Volume zu finden
- Vergleichbar mit der Inode Allocation Map von Unix-FS
- Für diesen gibt es einen MFT-Eintrag $BOOT.
- Sektor 0 enthält sämtliche Metadateninformationen zum Dateisystem
- zusätzlich findet sich hier der Bootcode zum hochfahren des Windows-Systems

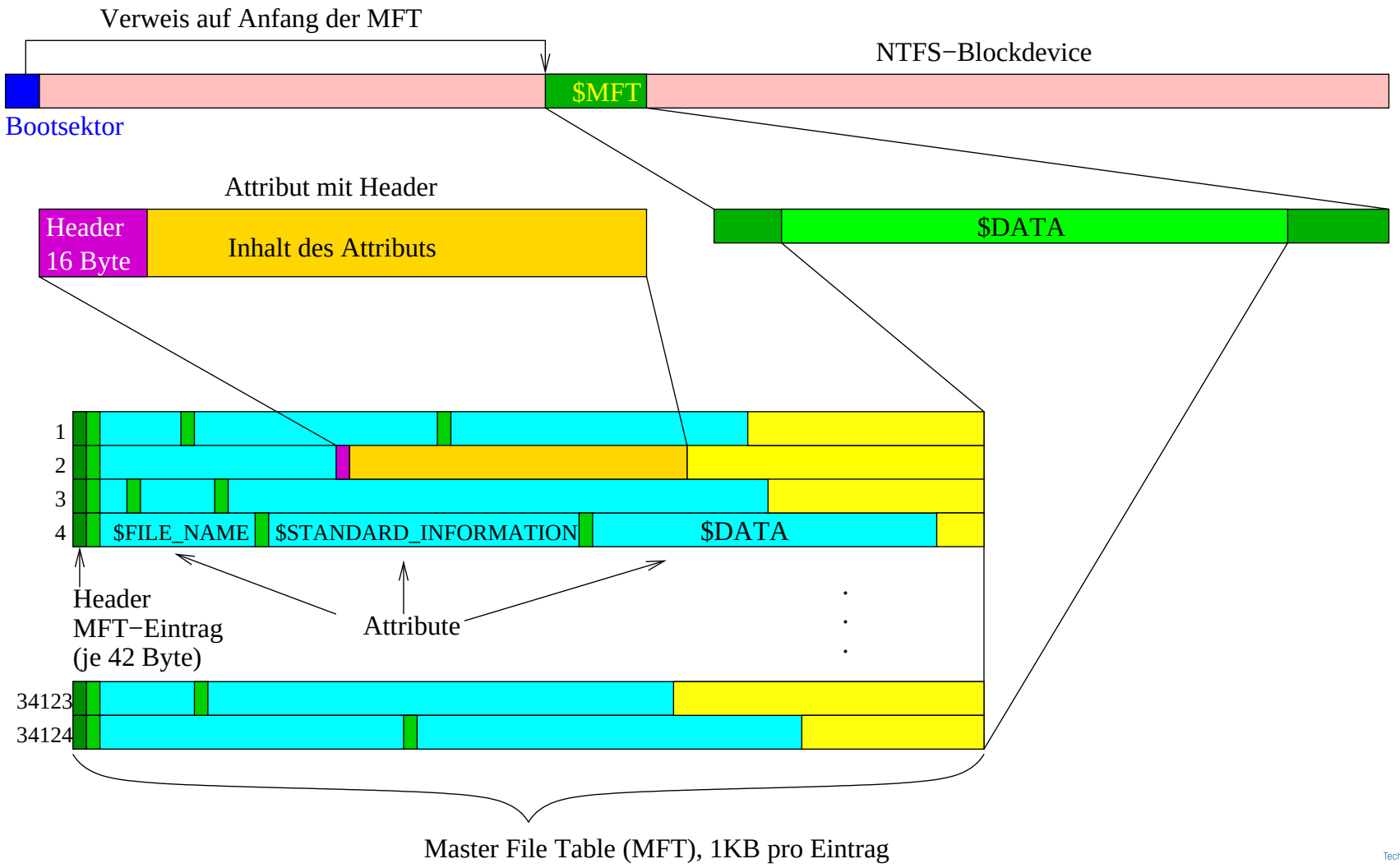
Auslesen ist mit dem Sleuthkitkommando fsstat möglich

Zahlreiche Attribute in NTFS

Was ist eine Datei?

- Alles ist eine Datei, dies ist allerdings was anderes als in Unixdateisystemen
- Eine NFTS-Datei ist eine Aneinanderreihung von Attributen
  - Das wichtigste: $DATA, der Dateiinhalt
  - Dazu viele weitere für Besitzer, Dateiname, Zugriffsrechte und -zeiten
- Es ist auch möglich, Attributtypen zu ändern!
- Im folgenden zeigen wir kurz die vorhandenen Standardattribute

Struktur eines einfachen MFT-Eintrags



Residente und Non-residente Attribute

Residente Attribute

- Attribut passt vollständig in den MFT-Eintrag. und $INDEX_ROOT.

Non-Residente Attribute

- Nur Header und Referenz auf Speicherort im MFT-Eintrag (als Extenttabelle bzw. Run List)
- z.B. ist $DATA oft non-resident. Warum ?
- Ähnlich bei Unixfs, Reiserfs

Weitere Eigenschaften der MFT

MFT-Eintrag reicht nicht für alle Attribute aus

- Zweiter MFT-Eintrag wird erstellt
- Erster erhält einen Verweis darauf

Benannte Attribute (Named Attributes)

- NTFS-Attribute verfügen oft noch über Name
- Ausnahme z.B. das $DATA-Attribut falls es das einzige in der Datei ist
- Das Attribut $LoggedUtilityStream heißt z.B. $EFS

Standardattribute (1)

Nr	Name
	Beschreibung
16	$STANDARD_INFORMATION
	Allg. Informationen
32	$ATTRIBUTE_LIST
	welche anderen Dateiattribute gibt es
48	$FILE_NAME
	UTF16-Dateiname und anderes
64	$OBJECT_ID
	Optional: eindeutige 64-Byte-File- bzw. Dir-ID
80	$SECURITY_DESCRIPTOR
	Dateizugriffskontrolle (wg. alten NTFS)

Standardattribute (2)

Nr	Name
	Beschreibung
96	$VOLUME_NAME
	Volumename (nur für $Volume Metadatenfile)
112	$VOLUME_INFORMATION
	Dateisystemversion und andere Flags
128	$DATA
	Inhalt der Datei
144	$ATTRIBUTE_ROOT
	Meta-Attribut Root-Node eines Indexbaums
160	$INDEX_ALLOCATION
	Meta-Attribut Node eines Indexbaums

Standardattribute (3)

Nr	Name
	Beschreibung
176	$BITMAP
	Bitmatp der $MFT-Datei und für andere Indexe
192	$REPARSE_POINT
	Daten über einen Reparse-Point
208	$EA_INFORMATION
	für Rückwärtskompatibilität von OS2 (HPFS)
224	$EA
	für Rückwärtskompatibilität von OS2 (HPFS)
256	$LOGGED_UTILITY_STREAM
	Schlüssel und Attribute über EFS-Attribute ($EFS)

Zeitstempel in $STANDARD_INFORMATION

Analog POSIX

- Zeit der letzten Änderung (File Altered Time) entspricht Mtime
- Zeit der letzten Änderung der Metadaten (MFT Altered Time) entspricht Ctime
- Zeit des letzten Zugriffs (File Acces) entsprecht der Atime (Access Time)

Keine Entsprechung in POSIX

Zeitpunkt der Erzeugung (Creation Time)

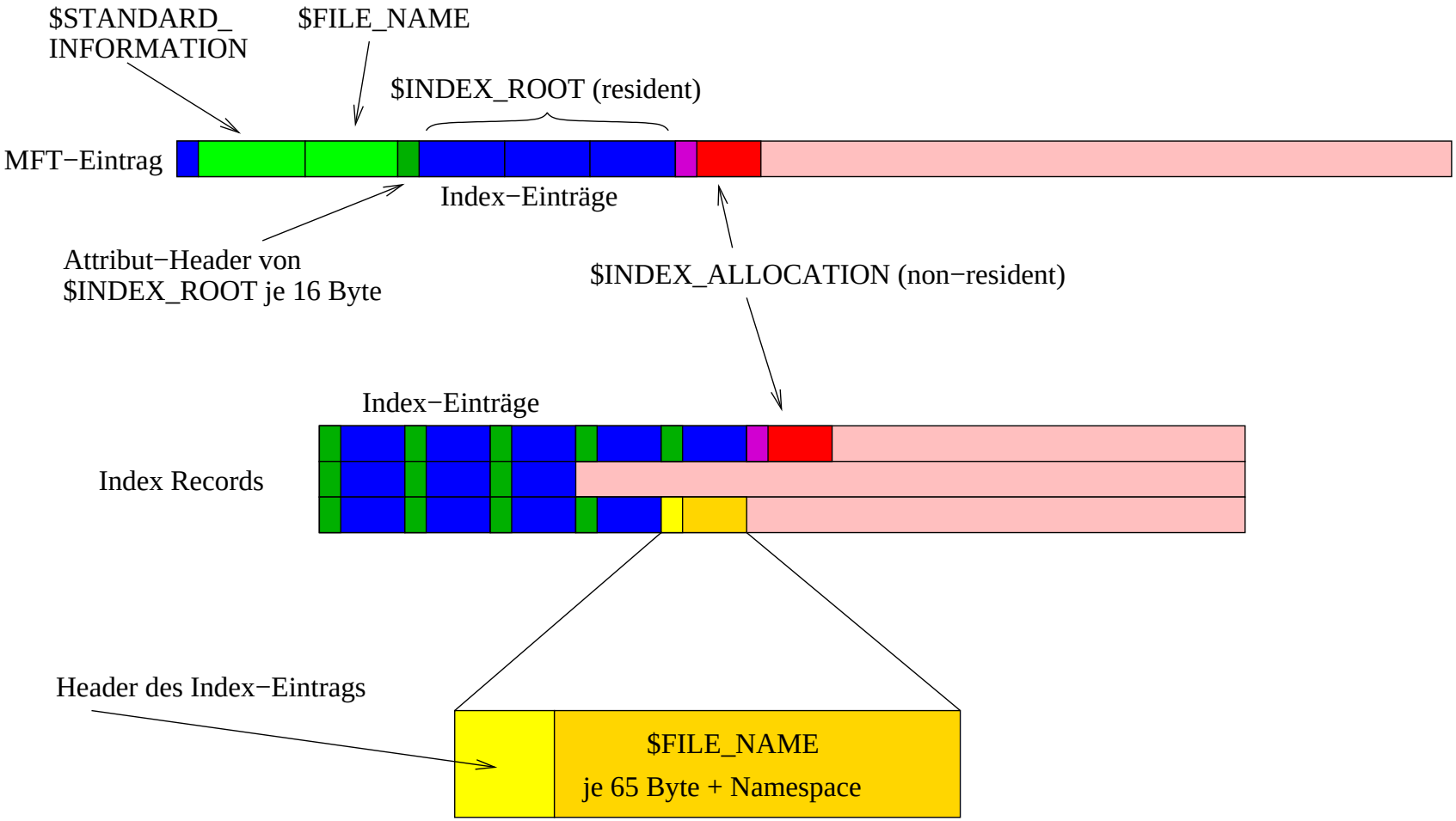
Datei-Attribute in $STANDARD_INFORMATION

- Die Datei-Flags (= Dateiattribute im Explorer) weisen nur darauf hin, daß es in der MFT überhaupt Attribute mit diesen Flags gibt
- Die Flags tauchen ebenso in den Attributheadern auf
- Damit können z.B. die Flags Sparse, Compressed und Encrypted für jedes Attribut einzeln gesetzt werden.
- Damit z.B. im Prinzip auch Dateimetainformation verschlüsselbar, allerdings macht Windows aktuell keinen Gebrauch davon
- Bedeutung dieser Flags bisher nur für $DATA

Verzeichnisse in NTFS

- Kennzeichnung eines Verzeichnisses redundant durch Flags
  - im Header des MFT-Eintrags
  - und im Attribut $FILE_NAME
- Verzeichnis ist analog Unix eine Tabelle von Datei-INode-Nummer-Paaren (hier: MFT-Referenznr.)
- Tabelle ist als B-Baum organisiert (kein B+-Baum wie in MS-Doku angegeben)
- Baumstruktur realisiert über die beiden Meta-Attribute $INDEX_ROOT und $INDEX_ALLOCATION
- Diese nehmen andere Attribute auf und kapseln diese
- Weitere Inhalte: Security-Infors, Listen der Reparsepoints u.a.

NTFS-Implementierung eines Verzeichnisses



Eigenschaften der Namensräume

DOS 8.3

- 8 Zeichen für Dateinamen
- 3 Zeichen für Suffix

Win32

- Lange Dateinamen mit Unicode
- Punkte und Leerzeichen im Dateinamen
- führender Punkt zugelassen

Posix

- Punkte am Ende möglich
- "Case Sensitive"
- Leerzeichen am Ende möglich

- Für jeder Datei gibt es für jeden Namensraum einen Verweis
- (ähnlich den Hardlinks unter Unix)
- Beispiel: `ntfsls -lis -d /dev/sda1`

```
 5917    2682834944 Feb 22 11:24 hiberfil.sys
    4        2560 Nov 17 01:03 $AttrDef
    8           0 Nov 17 01:03 $BadClus
    6     3698968 Nov 17 01:03 $Bitmap
    7        8192 Nov 17 01:03 $Boot
   11           0 Nov 17 01:03 $Extend
    2    67108864 Nov 17 01:03 $LogFile
    0    77398016 Nov 17 01:03 $MFT
    1        4096 Nov 17 01:03 $MFTMirr
    9           0 Nov 17 01:03 $Secure
   10      131072 Nov 17 01:03 $UpCase
    3           0 Nov 17 01:03 $Volume
15837           0 Nov 17 01:36 AUTOEXEC.BAT
11992         211 Nov 17 02:29 boot.ini
10739        4952 Aug  4 14:00 bootfont.bin
33426       26100 Feb 22 14:36 checkrun.txt
15836           0 Nov 17 01:36 CONFIG.SYS
11998           0 Nov 17 01:29 Dokumente und Einstellungen
```

- Das gleiche für DOS 8.3:

```
ntfsls -lisx -d /dev/sda1
```

```
 5917    2682834944 Feb 22 11:24 hiberfil.sys
    4        2560 Nov 17 01:03 $AttrDef
    8           0 Nov 17 01:03 $BadClus
    6     3698968 Nov 17 01:03 $Bitmap
    7        8192 Nov 17 01:03 $Boot
   11           0 Nov 17 01:03 $Extend
    2    67108864 Nov 17 01:03 $LogFile
    0    77398016 Nov 17 01:03 $MFT
    1        4096 Nov 17 01:03 $MFTMirr
    9           0 Nov 17 01:03 $Secure
   10      131072 Nov 17 01:03 $UpCase
    3           0 Nov 17 01:03 $Volume
15837           0 Nov 17 01:36 AUTOEXEC.BAT
11992         211 Nov 17 02:29 boot.ini
10739        4952 Aug  4 14:00 bootfont.bin
33426       26100 Feb 22 14:36 checkrun.txt
15836           0 Nov 17 01:36 CONFIG.SYS
11998           0 Nov 17 01:29 DOKUME~1
```

- z.B. "Dokumente und Einstellungen":
  `ntfsinfo -i 11998 -d /dev/sda1`
- Zunächst die allgemeinen Infos:

```
Dumping Inode #11998
Update Sequence Array Count:      3
$LogFile seqNum for this Inode:   0x1165a7435
Number of times reused:           1
Number of hard links:             2
MFT record Flags:                 DIRECTORY
Size - Used:                      624 bytes
Size - Allocated:                 1024 bytes
Next Attribute Instance Num       7
```

... dann das Attribut $STANDARD_INFORMATION

```
Dumping attribute $STANDARD_INFORMATION (0x10)
 File Creation Time:       Wed Nov 17 01:29:53 2004
 File Altered Time:        Sun Apr 17 19:53:11 2005
 MFT Changed Time:         Sun Apr 17 19:53:11 2005
 Last Accessed Time:       Mon Oct  9 20:22:52 2006
 Max Number of Versions:   0
 Version Number:           0
 Class ID:                 0
 User ID:                  0
 Security ID:              265
```

... danach der Dos-Filname (1. Hardlink)

```
Dumping attribute $FILE_NAME (0x30)
 File Name:                    'DOKUME~1'
 File Name Length:             8
 Allocated File Size:          0
 Real File Size:               0
 File Creation Time:           Wed Nov 17 01:29:53 2004
 File Altered Time:            Wed Nov 17 01:29:53 2004
 MFT Changed Time:             Wed Nov 17 01:29:53 2004
 Last Accessed Time:           Wed Nov 17 01:29:53 2004
```

... danach der lange Dateiname (2. Hardlink)

```
Dumping attribute $FILE_NAME (0x30)
 File Name:                    'Dokumente und Einstellungen'
 File Name Length:             27
 Allocated File Size:          0
 Real File Size:               0
 File Creation Time:           Wed Nov 17 01:29:53 2004
 File Altered Time:            Wed Nov 17 01:29:53 2004
 MFT Changed Time:             Wed Nov 17 01:29:53 2004
 Last Accessed Time:           Wed Nov 17 01:29:53 2004
```

```
Dumping attribute $INDEX_ROOT (0x90)
 Index name:                    '$I30'
 Indexed Attr Type:             file names
 Collation Rule:                1
 Index Block Size:              4096
 Clusters Per Block:            1
 Allocated Size:                40
 Used Size:                     40
 Flags:                         0x01
```

```
Dumping attribute $INDEX_ALLOCATION (0xA0)
 Index name:                 '$I30'
 Allocated data size:        4096
 Used data size:             4096
Dumping attribute $BITMAP (0xB0)
 Bitmap name:                '$I30'
 Bitmap's size:              8 bytes
End of inode reached
```

Gelöschte Verzeichniseinträge

- Das Sleuthkitaufruf `fls -d -f ntfs /dev/sda1` zeigt auch gelöschte Verzeichniseinträge

```
d/d * 12466-144-7(realloc):     Programme
r/d * 40046-144-5(realloc):     recorder.txt
d/d * 21741-144-1(realloc):     RECYCLER
d/d * 11996-144-6(realloc):     System Volume Information
r/r * 40776-128-5(realloc):     tasktimes.txt
r/r * 38710-128-1(realloc):     Testfile0
d/d * 11709-144-6(realloc):     WINDOWS
d/d * 11709-144-6(realloc):     WINDOWS
d/d * 11709-144-6(realloc):     WINDOWS
```

Typ Zeitstempel	Datei							
	umbenennen	lokal verschieben	in Volume verschieben	kopieren	zugriff	modifizieren	erstellen	löschen
$STD Info mtime						X	X	
$STD Info atime			X	X	V7	X	X	
$STD Info ctime				X			X	
$STD Info MFT mtime	X	X	X	X			X	X
Filename mtime		U		X			X	U
Filename atime			X	X			X	
Filename ctime				X			X	
Filename MFT mtime		U		X			X	U

U: Update gemäß $STD Info; V7: Keine Änderung bei Vista/Windows7; X: Änderung

Verkettungen in NTFS

- NTFS kennt Hardlinks und Softlinks
- Hardlinks kennen wir bereits: Mehrere Einträge für die gleiche Inode
- Softlinks auf Applikationsebene: LNK-Dateien
- Softlinks auf Verzeichnisse auf Dateisystemebene heißen bei NTFS Junctions
- NTFS-Struktur dahinter sind die sogenannten Reparsepoints
- Falls dazugehöriges Flag in $STANDARD_INFORMATION und $FILE_NAME gesetzt ist, liefert $REPARSE_POINT das Linkziel
- Vorsicht: Reparsepoint wird auch für Mountpoints des DFS (Distributed File System), für HSM (Hierarchical Storage Management) und anderes verwendet.

Alternative Datastreams

- Inhalt einer NTFS-Datei mit Linuxtools nicht immer einfach eindeutig bestimmbar
- Datei kann ja mehrere Attribute vom Typ $DATA besitzen
- Windows verwendet diese für Zusatzinformationen wie:
  - Autor, Kategorie, Thema etc.
  - also alle Infos, die der Explorer in der erweiterten Anzeige zeigt
- Zugriff unter Linux mit dem bereits bekannten nftsinfo
- Das Sleuthkit-Kommando icat -f ntfs /dev/hda1 432-128-5 liefert auch die einzelnen Streams
- Als Argument wird MFT-Referenznummer-Attributtyp-Attributinstanz verwendet.

ntfsundelete

- `ntfsundelete` ist ebenfalls ein Tool aus dem Paket ntfsprgs
- Es verfügt über drei Operationsmodi:
  - scan sucht nach gelöschten Dateien und gibt Name, Größe und Inodenummer aus.
    Zeigt an, ob und wieviel von der Datei wiederhergestellt werden kann.
    Einsatz verschiedener Filter möglich: z.B. alle Dateien, die seit gestern gelöscht wurden,
  - undelete nimmt Inode und stellt möglichst viele Daten der Datei wieder her und kopiert sie woanders hin.
    Es findet kein Schreibzugriff auf die NTFS-Partition statt
  - copy kopiert einen Teil der MFT für weitere Analysen in eine Datei.
- Immer mit eigenem Image und Loopbackdevice arbeiten!

Anwendung von ntfsundelete

- `ntfsundelete -s /dev/loop0`

```
Inode     Flags  %age  Date            Size  Filename
-----------------------------------------------------------------
16        F..!     0%  1970-01-01        0  <none>
17        FN..     0%  2005-08-15     3478  bootex.log
18        F..!     0%  1970-01-01        0  <none>
...
70946     FN..   100%  2005-11-24     1192  100000143143_a_050[1].jpeg
70947     FN..   100%  2005-11-24     1549  100000100887_a_050[1].jpeg
70948     FN..   100%  2005-11-24     1660  100000143046_a_050[1].jpeg
70949     FN..     0%  2005-11-24     1466  100000100889_a_050[1].jpeg
70950     FN..     0%  2005-11-24     2841  100000118205_a_112[1].jpeg
70951     FN..     0%  2005-11-24     3140  100000143143_a_112[1].jpeg
70952     FN..    50%  2005-11-24     4263  100000100887_a_112[1].jpeg
...
```

Schattenkopien unter Windows

- Systemwiederherstellungsmodus ab Windows Vista gegenüber Windows XP stark erweitert
- XP überwacht nur bestimmte Dateitypen
- Vista erstellt Schnappschüsse
  - der kompletten Systempartition
  - weitere vom Anwender bestimmte Laufwerke
- Dazu werden sogenannte Schattenkopien erzeugt (Volume Shadow Copy Service, kurz VSS)
- Technik zwar auch schon bei XP und Windows Server 2003 vorhanden, aber erst in Vista für Wiederherstellungspunkte genutzt
- Versionsstände nur täglich, mehrmalige Änderungen am Tag gehen verloren

Schattenkopien seit Windows Vista (2)

- Durch Filtertreiber `volsnap.sys` realisiert
- Treiber arbeitet auf Blockebene, trotzdem bietet Vista den Service nur für NTFS an
- Treiber kontrolliert alle Schreibzugriffe per „Copy on Write“-Strategie, d.h. alter Blockinhalt wird vor Freigabe auf Schattenkopiespeicher ausgelagert
- Schattenkopien bauen inkrementell aufeinander auf
- Per Default 15% Speicherplatz für Schattenkopien reserviert
- Bei Platzmangel wird ältester Inhalt überschrieben
- Schattenkopien bei Versionen Ultimate, Business und Enterprise über Menue in Dateieigenschaften wiederherstellbar
- Aber: Auch die billigeren Versionen führen Schattenkopien - was aber den meisten Anwendern unbekannt ist

Literaturempfehlungen I

- Alexander Geschonneck
  Computer Forensik - dpunkt.verlag, 2. Auflage 2006
- Keith J. Jones, Richard Bejtlich, Curtis W. Rose
  Real Digital Forensics - Addison-Wesley, 2006
- Steve Bunting, Wiliam Wei
  EnCase Computer Forensics, Wiley Publishing 2006.
- Chad Steel
  Window Forensics - The Field Guide for Conducting Corporate Computer Investigations, Wiley Publishing 2006.
- Dan Farmer, Wietse Venema
  Forensic Discovery. Addison-Wesley, 2005

Literaturempfehlungen II

Jon Erickson
Hacking - The Art of Exploitation, 2ND Edition, No Starch Press, 2008.

Lorenz Kuhlee und Victor Völzow
Computer Forensik Hacks O´Reilly Hacks Series, 2012.

Marc Ruel
Die Kunst des Penetration Testing - Handbuch für professionelle Hacker, Computer & Literatur Verlag, 2007.

Anonymous
Hacker's Guide, Markt+Technik Verlag 2004

Literaturempfehlungen III

Harlan Carvey
Perl Scripting for Windows Security: Live Response, Forensic Analysis and Monitoring, Elsevier, 2007.